

C-Multi-UR-AG

Yongcheng Song* Xinyi Huang Rongmao Chen Jiang Zhang Debiao He
Xiaofeng Chen Junzuo Lai Yi Wang Chao Lin Jiang Tan
Kang Lin

June 30, 2026

C-Multi-UR-AG is a code-based KEM running for standardization to ICCS’s NGCC competition in the category “public-key cryptographic algorithms”. Parameters sets are given for the three categories: 128, 256, and 512-bit security. The main features are:

- IND-CCA2 KEM
- Unstructured decoding assumptions
- Small bandwidth among code- and lattice-based unstructured KEMs
- Precise decryption failure rate
- Efficient implementations based on classical decoding algorithms
- Clean and clear structure

Table 1: Sizes, DFR, classical/quantum security strength, and space complexity

Instance	sks	pks	cts	total	$ K $	DFR	Classical Security Strength	Quantum Security Strength	Space Complexity
Level-128	94	3866	7332	11198	64	2^{-148}	2^{170}	2^{85}	2^{71}
Level-256	112	10780	15304	26084	64	2^{-268}	2^{276}	2^{138}	2^{89}
Level-512	155	28866	40926	69792	64	2^{-598}	2^{530}	2^{265}	2^{98}

sks: the private key size. pks: the public key size. cts: the ciphertext size. total = pks + cts. $|K|$: the shared key size. All sizes are in bytes. DFR: decapsulation failure rate. The classical security strengths are determined by the MM attack (see Table 6). The quantum security strengths are the square root of the classical security strengths. The space complexity corresponds to the classical security strength.

*Contact: Yongcheng Song (yongchengsong@outlook.com)

Contents

1	Design Rationale and Changes	3
2	Notations and Preliminaries	3
2.1	Notations	3
2.2	Linear Codes with Rank Metric	4
2.3	The RSD and RSL Problems	4
2.4	Public Key Encryption	6
3	Specifications	7
3.1	Objects and Hash Functions	7
3.2	Augmented Gabidulin Codes and Decoding Algorithms	7
3.3	Public Key Encryption (C-Multi-UR-AG.PKE)	11
3.4	Key Encapsulation Mechanism (C-Multi-UR-AG.KEM)	12
4	Parameters	13
5	Performance Analysis	14
5.1	Reference Implementation	15
5.2	Optimized Implementation	15
6	Security Proof	15
7	Known Attacks on the RSD Problem	18
7.1	Combinatorial Attacks	18
7.2	Algebraic Attacks	22
7.3	Resistance to Quantum Attacks	26
7.4	Estimated Security Strength	26
8	Side-Channel Attacks	29
9	Advantages and Limitations	29
	References	30

1 Design Rationale and Changes

Our C-Multi-UR-AG (Compact Multi-UR-AG) is an improvement over Multi-UR-AG. The details of both algorithms and the changes made are as follows.

Multi-UR-AG: Multi-UR-AG, a code-based cryptosystem using rank metric codes, is an IND-CPA PKE proposed in IEEE TIT 2024 [16]. Multi-UR-AG is short for Multiple-syndromes Unstructured RQC with AG codes. Its main advantage is that its security is entirely based on random *unstructured* rank decoding problems [16]. In Multi-UR-AG, the Augmented Gabidulin (AG) codes [16] are used to encode a plaintext and decrypt a ciphertext. For the AG codes in [16], an efficient decoding algorithm should be developed for an efficient implementation of Multi-UR-AG, and a concise Decoding Failure Rate (DFR) should be derived for compact parameters of Multi-UR-AG.

C-Multi-UR-AG: We introduce several modifications on the AG codes to Multi-UR-AG to obtain Compact Multi-UR-AG, leading to an efficient implementation and a significant parameter reduction:

- **WBL decoding algorithm.** We employ the Berlekamp-Welch like (WBL) algorithm to decode AG codes. As shown in [38], AG codes can be efficiently decoded using the WBL algorithm [7, 31], achieving an arbitrarily low DFR and an efficient implementation.
- **Concise DFR.** In [38], we derive a concise DFR formula. This formula allows a flexible and compact choice of parameters when applying the AG codes to cryptosystems.

2 Notations and Preliminaries

In this section, we define some notations, and recall $\mathbb{F}_{q^m}$ -linear codes, the rank decoding problem, and the rank support learning problem.

2.1 Notations

- We denote by $\mathbb{N}$ the set of non-negative integer numbers, q prime or prime power, and $\mathbb{F}_{q^m}$ an extension of degree m of the finite field $\mathbb{F}_q$. We view $\mathbb{F}_{q^m}$ as an m -dimensional vector space over $\mathbb{F}_q$. Let $\alpha \in \mathbb{F}_{q^m}$ be a primitive element such that $\boldsymbol{\alpha} = (1, \alpha, \dots, \alpha^{m-1}) \in \mathbb{F}_{q^m}^m$ is a basis of $\mathbb{F}_{q^m}$ over $\mathbb{F}_q$.
- Vectors (resp. matrices) are denoted by lower-case (resp. upper-case) bold letters.
- We use the notation $[N] := \{1, 2, \dots, N\}$ for the first N natural numbers. For two positive integers N_1 and N_2 ($N_1 \leq N_2$), the notation $\{N_1..N_2\}$ represents positive integers from N_1 to N_2 .
- The term $\text{negl}(\lambda)$ denotes a negligible function in a security parameter λ .
- Let $\mathcal{S}$ be a finite set or a distribution. $s \stackrel{\$}{\leftarrow} \mathcal{S}$ denotes that s is sampled from $\mathcal{S}$ by uniform and random way.
- Let $\mathcal{A}$ be an algorithm. We say that $\mathcal{A}$ is a PPT algorithm if it is a probabilistic polynomial-time algorithm.
- Let $\mathbf{G}$ be a game. $\mathbf{G} \Rightarrow b$ means that $\mathbf{G}$ outputs b .

2.2 Linear Codes with Rank Metric

Definition 2.1 (Rank Metric). For $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{F}_{q^m}^n$, each coordinate x_i is associated to a vector of $\mathbb{F}_q^m$ w.r.t. the basis α . Then $\mathbf{x}$ is associated to an $m \times n$ matrix given by $\text{Mat}(\mathbf{x}) = (x_{ij})_{i \in [m], j \in [n]} : \mathbf{x} = \alpha \text{Mat}(\mathbf{x})$. The rank weight $\|\mathbf{x}\|_R$ of $\mathbf{x}$ is defined as the rank of $\text{Mat}(\mathbf{x})$. The rank distance $d_R(\mathbf{x}, \mathbf{y})$ between $\mathbf{x}$ and $\mathbf{y}$ in $\mathbb{F}_{q^m}^n$ is defined by $d_R(\mathbf{x}, \mathbf{y}) := \|\mathbf{x} - \mathbf{y}\|_R$.

Let $\langle x_1, x_2, \dots, x_n \rangle_{\mathbb{F}_q}$ be the $\mathbb{F}_q$ -linear subspace of $\mathbb{F}_{q^m}$ generated by linear combinations over $\mathbb{F}_q$ of coordinates of $\mathbf{x}$. The support $\text{Supp}(\mathbf{x})$ of $\mathbf{x}$ is defined as the $\mathbb{F}_q$ -linear subspace $\langle x_1, x_2, \dots, x_n \rangle_{\mathbb{F}_q}$, i.e., $\text{Supp}(\mathbf{x}) = \langle x_1, x_2, \dots, x_n \rangle_{\mathbb{F}_q}$. It follows from definition that $\|\mathbf{x}\|_R = \dim(\text{Supp}(\mathbf{x}))$. The set of such errors of weight r and length n is denoted by $\mathcal{S}_r^n$.

Support and Coefficient Matrices of the Error. For an error $\mathbf{e} \in \mathcal{S}_r^n$, let $\boldsymbol{\varepsilon} = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_r) \in \mathbb{F}_{q^m}^r$ be a basis of $\text{Supp}(\mathbf{e})$, then there exists a matrix $\mathbf{C} \in \mathbb{F}_q^{r \times n}$ of rank r such that $\mathbf{e} = \boldsymbol{\varepsilon} \mathbf{C}$. Under the basis α , there exists a matrix $\mathbf{S} \in \mathbb{F}_q^{m \times r}$ of rank r such that $\boldsymbol{\varepsilon} = \alpha \mathbf{S}$. Then $\mathbf{e} = \alpha \mathbf{S} \mathbf{C}$. We call $\mathbf{S}$ and $\mathbf{C}$ respectively support matrix and coefficient matrix.

Definition 2.2 ($\mathbb{F}_{q^m}$ -Linear Codes with Rank Metric). An $\mathbb{F}_{q^m}$ -linear code embedded with rank metric of length n and dimension k is a subspace of dimension k of $\mathbb{F}_{q^m}^n$. Such $\mathbb{F}_{q^m}$ -linear codes are denoted by $[n, k]_{q^m}$.

Given an $[n, k]_{q^m}$ -linear code $\mathcal{C}$, a matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ is called *generator matrix* iff $\mathcal{C} = \{\mathbf{m} \mathbf{G} : \mathbf{m} \in \mathbb{F}_{q^m}^k\}$ and a matrix $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$ is called *parity-check matrix* iff $\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_{q^m}^n : \mathbf{H} \mathbf{c}^\top = \mathbf{0}\}$. The systematic forms of $\mathbf{G}$ and $\mathbf{H}$ are respectively defined as $[\mathbf{I}_k \ \mathbf{P}]$ and $[-\mathbf{P}^\top \ \mathbf{I}_{n-k}]$ where $\mathbf{P} \in \mathbb{F}_{q^m}^{k \times (n-k)}$.

2.3 The RSD and RSL Problems

The security of code-based cryptosystems is based on the hardness of decoding assumptions. The decoding assumptions can be expressed by the generator and parity-check matrices, respectively. In the rank metric setting, we call the Rank Decoding (RD) problem and the Rank Syndrome Decoding (RSD) problem, respectively. Their computational versions are as follows.

Rank Decoding (RD) Problem: Let $\mathbf{G}$ be a generator matrix of a random $[n, k]_{q^m}$ -linear code $\mathcal{C}$, $\mathbf{y} \in \mathbb{F}_{q^m}^n$, and $r \in \mathbb{N}$. The problem is to find $\mathbf{x} \in \mathbb{F}_{q^m}^k$ and $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{y} = \mathbf{x} \mathbf{G} + \mathbf{e}$.

Rank Syndrome Decoding (RSD) Problem: Let $\mathbf{H}$ be a parity-check matrix of a random $[n, k]_{q^m}$ -linear code $\mathcal{C}$, $\mathbf{s} \in \mathbb{F}_{q^m}^{n-k}$, and $r \in \mathbb{N}$. The problem is to find $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{s} = \mathbf{H} \mathbf{e}^\top$.

The hardness of two problems is equivalent. Even if the RD problem is not known to be NP-complete, there is a randomized reduction [27] from the RD problem to an NP-complete problem, namely to decoding in the Hamming metric [14]. An RD instance can also be viewed as a structured MinRank instance; more precisely, there exists a general reduction from any RD instance to the MinRank instance [23]. The MinRank problem was proven NP-complete in [18].

The Rank Support Learning (RSL) problem consists of N distinct RSD instances whose errors share the same support. The RSL problem is related to homogeneous matrices. This matrix notion is the extension of the support and weight definitions of vectors to matrices.

Definition 2.3 (Homogeneous Matrix). A matrix $\mathbf{E} \in \mathbb{F}_{q^m}^{n \times N}$ is a *homogeneous matrix* if its entries span an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$ of dimension r , i.e., $\dim(\text{Supp}(\mathbf{E})) = \|\mathbf{E}\|_R = r$.

We denote the set of such homogeneous matrices by $\mathcal{S}_r^{n \times N}$:

$$\mathcal{S}_r^{n \times N} = \left\{ \mathbf{E} \in \mathbb{F}_{q^m}^{n \times N} : \dim(\text{Supp}(\mathbf{E})) = \|\mathbf{E}\|_R = r \right\}.$$

We here present the equivalent variant of the RSL problem, where we require that one recover the whole homogeneous matrix, instead of its support.

Definition 2.4 (RSL Problem). Let $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$ be a parity-check matrix of a random $[n, k]_{q^m}$ -linear code and $\mathbf{S} \in \mathbb{F}_{q^m}^{(n-k) \times N}$. The RSL problem is to find a matrix $\mathbf{E} \in \mathcal{S}_r^{n \times N}$ such that $\mathbf{H}\mathbf{E} = \mathbf{S}$.

We denote the RSL problem by $\text{RSL}_{q,m,n,k,r,N}$.

Definition 2.5 (Decisional RSL (DRSL) Problem). Let $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$ be a parity-check matrix of a random $[n, k]_{q^m}$ -linear code. The decisional RSL problem is hard, if for any PPT adversary $\mathcal{B}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,n,k,r,N}}(\lambda) := \left| \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{S}) = 1 : \mathbf{S} \xleftarrow{\$} \mathbb{F}_{q^m}^{(n-k) \times N} \right] - \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{H}\mathbf{E}) = 1 : \mathbf{E} \xleftarrow{\$} \mathcal{S}_r^{n \times N} \right] \right|.$$

The hardness of the RSL and DRSL problems is analyzed in [8, 16, 22, 25]:

- In [25], the authors introduced the RSL problem and analyzed firstly the complexity of the combinatorial attack by adapting the AGHT attack. When $N \geq nr$, the RSL problem can be solved in polynomial time. For the case $1 < N < nr$, the authors reduced the RSL problem to the problem finding a codeword of weight r in a code of same length and dimension containing q^N words of the weight r , and conjectured that the complexity is reduced by a factor $q^{\alpha RN}$ ($\alpha = \frac{m}{n}, R = \frac{k}{n}$) compared to the case $N = 1$. The complexity gain in fact is lower due to the deep correlation of codewords. Moreover, the authors showed that the attacks on the decisional RSL problem remain the direct attacks on the computational RSL problem, and the decisional and computational versions have equivalent hardness.
- In [22], the authors showed that the condition $N \leq kr$ should be satisfied in order to avoid a subexponential time attack.
- In [8], the authors introduced the algebraic attacks on the RSL problem by adapting the MM modeling. In [16], the authors clarified the algebraic attacks proposed in [8] and showed that the attacks are valid only when $N > n - k - r$. Moreover, the work [16] decreased the threshold where the RSL problem starts to be solvable in polynomial time and showed that when $N > kr$, the RSL problem actually leads to a polynomial time attack.

Overall, under combinatorial attacks, the complexity of solving the RSL problem is reduced by at most N bits relative to that of solving the RSD problem; under algebraic attacks, the RSL problem is almost as hard as the RSD problem for a small number of syndromes N , typically $N < kr$ and $N < n - k - r$.

2.4 Public Key Encryption

Definition 2.6 (Public Key Encryption). A Public Key Encryption (PKE) scheme $\text{PKE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$ consists of four polynomial-time algorithms:

- $\text{Setup}(1^\lambda) \rightarrow \text{par}$: The setup algorithm takes the security parameter 1^λ as input and outputs global system parameters par . We assume that par implicitly defines a plaintext space $\mathcal{M} = \mathcal{M}_{\text{par}}$.
- $\text{KGen}(\text{par}) \rightarrow (\text{pk}, \text{sk})$: The key generation algorithm that takes par as input and outputs a public key pk and a private key sk .
- $\text{Enc}(\text{pk}, \text{m}; \theta) \rightarrow \text{ct}$: The encryption algorithm that takes pk , a plaintext $\text{m} \in \mathcal{M}$, and randomness θ as inputs and outputs a ciphertext ct .
- $\text{Dec}(\text{sk}, \text{ct}) \rightarrow \text{m} / \perp$: The decryption algorithm that takes sk and ct as inputs and outputs $\text{m} \in \mathcal{M}$ or a rejection symbol $\perp$.

Correctness. The PKE is $(1 - \delta)$ -correct, if for any $\text{par} \leftarrow \text{Setup}(1^\lambda)$ and any $\text{m} \in \mathcal{M}$, the following probability holds:

$$\Pr[\text{m} = \text{m}' : (\text{pk}, \text{sk}) \leftarrow \text{KGen}(\text{par}), \text{ct} \leftarrow \text{Enc}(\text{pk}, \text{m}; \theta), \text{m}' \leftarrow \text{Dec}(\text{sk}, \text{ct})] \geq 1 - \delta,$$

where the probability depends on the randomness of KGen and Enc , and δ is called the decryption failure rate.

Security Model. The basic security requirement for PKE is IND-CPA, Indistinguishability under Chosen Plaintext Attacks. An IND-CPA secure PKE is defined by the game $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda)$ between a challenger and a PPT adversary $\mathcal{A}$:

Game $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda)$

```

01  $\text{par} \leftarrow \text{Setup}(1^\lambda)$ 
02  $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(\text{par})$ 
03  $(\text{m}_0, \text{m}_1) \xleftarrow{\$} \mathcal{A}(\text{par}, \text{pk})$ 
04  $b \xleftarrow{\$} \{0, 1\}$ 
05  $\text{ct} \leftarrow \text{Enc}(\text{pk}, \text{m}_b; \theta)$ 
06  $b' \leftarrow \mathcal{A}(\text{par}, \text{pk}, \text{ct})$ 
07  $b' \neq b : \text{return } 0$ 
08 return 1

```

Figure 1: The IND-CPA security game for PKE $\text{PKE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$

Definition 2.7 (IND-CPA Security). A PKE scheme $\text{PKE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$ is IND-CPA secure if for any PPT adversary $\mathcal{A}$, its advantage is bounded by a negligible function $\text{negl}(\lambda)$:

$$\text{Adv}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \left| \Pr[\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

3 Specifications

In this section, we describe the scheme: C-Multi-UR-AG. We begin with the PKE version (C-Multi-UR-AG.PKE), then exploit the salted Fujisaki–Okamoto (FO) transformation [30] to obtain an IND-CCA2 KEM (C-Multi-UR-AG.KEM).

3.1 Objects and Hash Functions

Field elements. Elements of $\mathbb{F}_{q^m}$ are represented as vectors of size m over $\mathbb{F}_q$. In our setting, q is set to 2. Thus, any $e \in \mathbb{F}_{q^m}$ can be represented as $(e_0, e_1, \dots, e_{m-1}) \in \mathbb{F}_q^m$.

Vectors. Elements of $\mathbb{F}_{q^m}^n$ are represented as n -dimensional arrays over $\mathbb{F}_{q^m}$.

Polynomials. When $\mathbf{e}$ is viewed as a polynomial in $\mathcal{R}$, its first coordinate corresponds to the constant coefficient of $\mathbf{e}$.

Randomness. Application randomness is drawn from the deterministic random bit generator provided by the NGCC API, which is instantiated with SM3 and maintains a 55-byte internal state.

Seeds. The considered `seedexpander` has been provided by the NGCC. It is initialized with a byte string of length 64 and used to deterministically derive the key pair and the encryption randomness from a master seed.

Salts. A fresh salt is drawn in every `Encaps` call as a bit string of length 512, and its length is denoted by `len`. The salt is bound to the ciphertext to prevent multi-target attacks.

Hash Functions. The hash functions `G` and `H` are instantiated using the `pseudohash` construction provided by the NGCC API (built from SM3 and HMAC-SM3) with a 64-byte output. The output of `G` is used as a seed, and the output of `H` is the encapsulated key.

3.2 Augmented Gabidulin Codes and Decoding Algorithms

Gabidulin codes are rank metric codes and were proposed by Gabidulin [24]. A Gabidulin code is the evaluation of q -polynomials [34]. Among variants of Gabidulin codes [12, 16, 32], Augmented Gabidulin (AG) codes [16] are particularly promising. The AG codes can actually be traced back to the work [12] for its general equivalent form. The AG codes are still the evaluation of q -polynomials, but with a wide parameters region.

The q -polynomial is essentially the polynomial of the Frobenius automorphism over $\mathbb{F}_{q^m}$. We here recall the Frobenius automorphism from $\mathbb{F}_{q^m}$ to $\mathbb{F}_{q^m}$ for facilitating the understanding of the essence of q -polynomials. We denote it as:

$$\begin{aligned} \theta : \quad \mathbb{F}_{q^m} &\longrightarrow \mathbb{F}_{q^m} \\ x &\longmapsto \theta(x) := x^q. \end{aligned}$$

The Frobenius automorphism θ is an $\mathbb{F}_q$ -automorphism of $\mathbb{F}_{q^m}$. We use its following properties:

- $\forall x_1, x_2 \in \mathbb{F}_{q^m}, \beta_1, \beta_2 \in \mathbb{F}_q: \theta(x_1 x_2) = \theta(x_1) \theta(x_2), \theta(\beta_1 x_1 + \beta_2 x_2) = \beta_1 \theta(x_1) + \beta_2 \theta(x_2)$ (linearity over $\mathbb{F}_q$),
- $\forall i \in \mathbb{N}, \theta^i(x) = \theta^{i-1}(\theta(x)) = \theta^{i-1}(x^q) = \dots = \theta(x^{q^{i-1}}) = x^{q^i},$

- $\forall i \in \mathbb{N}$, θ^i is also a Frobenius automorphism. This is because the set of all Frobenius automorphisms is a cyclic group of order m (The identity is θ^0).

Let $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{F}_{q^m}^n$. The Frobenius automorphism θ can be applied to $\mathbf{x}$ coordinate-wise: $\theta(\mathbf{x}) = (\theta(x_1), \theta(x_2), \dots, \theta(x_n)) = (x_1^q, x_2^q, \dots, x_n^q)$. Further, for any $i \in \mathbb{N}$, $\theta^i(\mathbf{x}) = (\theta^i(x_1), \theta^i(x_2), \dots, \theta^i(x_n)) = (x_1^{q^i}, x_2^{q^i}, \dots, x_n^{q^i})$.

Definition 3.1 (q -Polynomials [34]). A q -polynomial of q -degree n over $\mathbb{F}_{q^m}$ is a polynomial of the form $f(x) = \sum_{i=0}^n \theta^i(x) = \sum_{i=0}^n f_i x^{q^i}$ for $f_i \in \mathbb{F}_{q^m}$, $f_n \neq 0$.

A q -polynomial in a variable x is essentially a polynomial of θ acting on x . The addition (resp. symbolic product “ $\circ$ ”) of two q -polynomials corresponds to the addition (resp. composition “ $\bullet$ ”) of two polynomials of θ . Let $a(x) = \sum_i a_i x^{q^i}$ and $b(x) = \sum_i b_i x^{q^i} \in \mathcal{L}[x]$. Let $a(\theta) = \sum_i a_i \theta^i$ and $b(\theta) = \sum_i b_i \theta^i$.

- $a(\theta)(x) = (\sum_i a_i \theta^i)(x) = \sum_i a_i \theta^i(x) = \sum_i a_i x^{q^i} = a(x)$,
- $(a(\theta) + b(\theta))(x) = (\sum_i (a_i + b_i) \theta^i)(x) = \sum_i (a_i + b_i) x^{q^i} = a(x) + b(x)$,
- $(a(\theta) \bullet b(\theta))(x) = a(b(\theta))(x) = a(b(x)) = a(x) \circ b(x)$.

We denote the q -degree of a q -polynomial $f(x)$ by $\deg_q f(x)$. Let $\mathcal{L}[x]$ be the set of q -polynomials and $\mathcal{L}_{\leq r}[x]$ be the set of q -polynomials of q -degree $\leq r$. Let $a(x) = \sum_i a_i x^{q^i}$ and $b(x) = \sum_i b_i x^{q^i} \in \mathcal{L}[x]$:

- The addition “ $+$ ” of $a(x)$ and $b(x)$: $a(x) + b(x) = \sum_i (a_i + b_i) x^{q^i}$,
- The symbolic product “ $\circ$ ” of $a(x)$ and $b(x)$: $a(x) \circ b(x) = a(b(x))$.

It is clear that the set of q -polynomials together with addition and symbolic product forms a noncommutative ring. In such a ring, the symbolic product is associative and distributive w.r.t. both right and left product. The identity element is $\theta^0(x) = x^{q^0} = x$ and there are no divisors of zero, i.e., $a(x) \circ b(x) = 0$ implies $a(x) = 0$ or $b(x) = 0$. A q -polynomial $c(x)$ is said to be symbolically right divisible by $b(x)$ if $c(x) = a(x) \circ b(x)$. When $c(x) = a(x) \circ b(x)$, we say that $c(x)$ is symbolically left divisible by $a(x)$, which is denoted by $b(x) = a(x) \setminus c(x)$.

A q -polynomial $f \in \mathcal{L}[x]$ satisfies:

- $\forall x_1, x_2 \in \mathbb{F}_{q^m}$, $\beta_1, \beta_2 \in \mathbb{F}_q$, $f(\beta_1 x_1 + \beta_2 x_2) = \beta_1 f(x_1) + \beta_2 f(x_2)$,
- If x_1 and x_2 are any two roots of f , then $f(x_1) = f(x_2) = f(x_1 + x_2) = 0$.

These two properties imply that all roots of a q -polynomial span an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$. Proposition 3.1 shows that the q -degree of a q -polynomial is lower bounded by the dimension of its root space. Definition 3.2 shows that given an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$, there exists a unique monic q -polynomial vanishing on such an $\mathbb{F}_q$ -subspace.

Proposition 3.1. Let $f(x) \in \mathcal{L}[x]$ be a q -polynomial. Assume that $e_1, e_2, \dots, e_n \in \mathbb{F}_{q^m}$ span an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$ of dimension r . If $f(e_i) = 0$ for all $i \in \{1..n\}$, then either $f(x) = 0$, or $\deg_q f(x) \geq r$.

Definition 3.2 (Annihilator Polynomial [34]). For any $\mathbb{F}_q$ -subspace E of $\mathbb{F}_{q^m}$ of dimension r , there exists a unique monic q -polynomial $\mathcal{A}_E(x)$ of q -degree r that vanishes on E , i.e., $\mathcal{A}_E(x) = 0$ for any $x \in E$. Such a monic q -polynomial is called the annihilator polynomial of E .

Since the coordinates of any error of weight r uniquely span a r -dimensional $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$, i.e., its r -dimensional support, we have Definition 3.3.

Definition 3.3 (Annihilator Polynomial of Vector). For any $\mathbf{e} = (e_1, e_2, \dots, e_n) \in \mathcal{S}_r^n$, let $\mathcal{A}_{\mathbf{e}}(x)$ be the annihilator polynomial of the support of $\mathbf{e}$. We call $\mathcal{A}_{\mathbf{e}}(x)$ the annihilator polynomial of $\mathbf{e}$. $\mathcal{A}_{\mathbf{e}}(x)$ is unique, has q -degree r , and vanishes on $\mathbf{e}$, i.e., $\mathcal{A}_{\mathbf{e}}(e_i) = 0$ for all $i \in \{1..n\}$.

Proposition 3.2 shows that the dimension of $\mathbb{F}_q$ -subspace spanned by some roots of a q -polynomial is upper bounded by its q -degree.

Proposition 3.2. Let $f(x)$ be a q -polynomial of q -degree r . Let $\{e_i : f(e_i) = 0, i \in \{1..n\}\}$ be a set consisting of any n roots of $f(x)$. Then the dimension of the $\mathbb{F}_q$ -subspace spanned by such a set is at most r . In other words, if $\mathbf{e} = (e_1, e_2, \dots, e_n)$, then $\|\mathbf{e}\|_{\mathbb{R}} \leq r$.

Definition 3.4 (Augmented Gabidulin (AG) Codes [13, 16, 38]). Let q, m, n, k, t be integers and $k \leq t \leq \min\{n, m\}$. Let $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ be the generator of weight t . An Augmented Gabidulin code of dimension k and length n generated by $\mathbf{g}$ is defined as

$$\text{AG}_{k,t}^{\mathbf{g}} = \left\{ f(\mathbf{g}) = (f(g_1), f(g_2), \dots, f(g_n)) : f(x) \in \mathcal{L}_{\leq k-1}[x] \right\}.$$

The concept of AG codes was introduced in [16], where Gabidulin codes are augmented by appending zero entries to their generators. An equivalent formulation [12] can be traced back to [13], in which the generator is linearly dependent, rather than containing zero entries. We adopt this equivalent form, which enabled [38] to develop efficient decodings and derive a concise DFR. The generator matrix of $\text{AG}_{k,t}^{\mathbf{g}}$ is

$$\mathbf{G} = \begin{bmatrix} g_1 & g_2 & \cdots & g_n \\ g_1^q & g_2^q & \cdots & g_n^q \\ \vdots & \vdots & \ddots & \vdots \\ g_1^{q^{k-1}} & g_2^{q^{k-1}} & \cdots & g_n^{q^{k-1}} \end{bmatrix}.$$

The $\text{AG}_{k,t}^{\mathbf{g}}$ codes in Definition 3.4 can decode errors of weight up to $\min\{t - k, \lfloor \frac{n-k}{2} \rfloor\}$.

Decoding AG Codes. There currently are three ways to decode AG codes: Support Erasure (SE) technique [16], Welch-Berlekamp Like (WBL) algorithm [38], and Gaussian eliminations [38]. The last two are essentially designed to solve the Linear Reconstruction (LR) problem arising from decoding AG codes. For our cryptographic parameters, the practical one is the WBL algorithm.

Given the AG codes in Definition 3.4, we assume that the message q -polynomial is $f(x) \in \mathcal{L}_{\leq k-1}[x]$, the error occurring at the channel is $\mathbf{e}$ and $\|\mathbf{e}\|_{\mathbb{R}} \leq r$, the received word is $\mathbf{y} = f(\mathbf{g}) + \mathbf{e}$. The aim of decoding AG codes is to recover $\mathbf{e}$ and $f(x)$ from $\mathbf{y}$. The decoding AG codes problem is as follows.

Definition 3.5 (Decoding AG Code Problem DecAGCode($\mathbf{g}, \mathbf{y}$)).

Input : $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_{\mathbb{R}} = t$; $\mathbf{y} = (y_1, y_2, \dots, y_n) \in \mathbb{F}_{q^m}^n$.

Output : $f(x) \in \mathcal{L}[x]$ and $\mathbf{e} \in \mathbb{F}_{q^m}^n$ such that: 1. $\deg_q f(x) \leq k-1$; 2. $\mathbf{e} \neq \mathbf{0}_n$ and $\|\mathbf{e}\|_{\mathbb{R}} \leq r$; 3. $\mathbf{y} = f(\mathbf{g}) + \mathbf{e}$.

From Definition 3.3, we know that for any $\mathbf{e} \in \mathcal{S}_r^n$, there exists a unique annihilator polynomial $\mathcal{A}_{\mathbf{e}}(x)$ of q -degree r that vanishes on $\mathbf{e}$, i.e., $\mathcal{A}_{\mathbf{e}}(e_i) = 0$ for all $i \in \{1..n\}$. Thus, it is natural to transform the DecAGCode($\mathbf{g}, \mathbf{y}$) problem into the Non Linear Reconstruction (NLR) problem (Definition 3.6).

Definition 3.6 (Non Linear Reconstruction Problem $\text{NLR}(\mathbf{g}, \mathbf{y})$).

Input : $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_{\mathbb{R}} = t$; $\mathbf{y} = (y_1, y_2, \dots, y_n) \in \mathbb{F}_{q^m}^n$.

Output : $f(x), v(x) \in \mathcal{L}[x]$ such that: 1. $\deg_q f(x) \leq k-1$; 2. $v(x) \neq 0$ and $\deg_q v(x) \leq r$; 3. $v(\mathbf{y}) = (v \circ f)(\mathbf{g})$.

Solving the NLR problem can be reduced to solving the LR problem in Definition 3.7.

Definition 3.7 (Linear Reconstruction Problem $\text{LR}(\mathbf{g}, \mathbf{y})$).

Input : $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_{\mathbb{R}} = t$; $\mathbf{y} = (y_1, y_2, \dots, y_n) \in \mathbb{F}_{q^m}^n$.

Output : $v(x), u(x) \in \mathcal{L}[x]$ such that: 1. $v(x) \neq 0$ and $\deg_q v(x) \leq r$; 2. $\deg_q u(x) \leq k+r-1$; 3. $v(\mathbf{y}) = u(\mathbf{g})$.

In [38], the $\text{NLR}(\mathbf{g}, \mathbf{y})$ and $\text{LR}(\mathbf{g}, \mathbf{y})$ problems are shown to be equivalent.

The WBL algorithm consists in solving the $\text{LR}(\mathbf{g}, \mathbf{y})$ problem, decomposed into two steps:

1. **Initialization step:** Two pairs of q -polynomials (u_0, v_0) and (u_1, v_1) satisfying

$$v_0(y_i) = u_0(g_i) \quad \text{and} \quad v_1(y_i) = u_1(g_i)$$

for $1 \leq i \leq k$ are computed. u_0 is set as the annihilator polynomial on $g_1, \dots, g_k$, u_1 is set as the q -polynomial interpolating $y_1, \dots, y_k$ on $g_1, \dots, g_k$, and

$$v_0(x) = 0 \quad \text{and} \quad v_1(x) = x.$$

2. **Interpolation step:** Iteratively, the q -degrees of (u_0, v_0) and (u_1, v_1) are increased using a recurrence relation ensuring that if the interpolation conditions

$$v(y_i) = u(g_i) \quad \text{for } 1 \leq i \leq j$$

are satisfied at step j , then they are also satisfied at step $j+1$. Besides, this construction ensures that at least one of the pairs satisfies the final degree conditions

$$\deg_q v(x) \leq r \quad \text{and} \quad \deg_q u(x) \leq k+r-1$$

when the q -polynomials are initialized according to the aforementioned Initialization step.

The correctness of the WBL algorithm is proved in [7]. Once the $\text{LR}(\mathbf{g}, \mathbf{y})$ problem is solved, one can solve $\text{DecAGCode}(\mathbf{g}, \mathbf{y})$ for AG codes as Algorithm 1. The DFR decoding errors of weight r is analyzed in [38].

Algorithm 1 Decoding errors of weight $\leq r$ by the WBL algorithm

Input: $q, m, n, k, r \in \mathbb{N}$, $\mathbf{g} \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_{\mathbb{R}} = t$, $\mathbf{y} \in \mathbb{F}_{q^m}^n$.

Output: $f(x) \in \mathcal{L}_{\leq k-1}[x]$; $\mathbf{e} \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{e}\|_{\mathbb{R}} \leq r$

- 1: Call the WBL algorithm from [7] (Algorithm 5)
 - 2: Return (N_1, W_1)
 - 3: Set $u(x) = N_1$ and $v(x) = W_1$
 - 4: Compute $f(x) = v(x) \setminus u(x)$
 - 5: **if** $\deg_q f(x) \leq k-1$ and $\|\mathbf{y} - f(\mathbf{g})\|_{\mathbb{R}} \leq r$:
 - 6: **return** $f(x)$ and $\mathbf{e} := \mathbf{y} - f(\mathbf{g})$
 - 7: **else:**
 - 8: **return** $\perp$
-

Theorem 3.1 (DFR for AG codes [38]). *Let $\mathbf{e} \in \mathbb{F}_{q^m}^n$ be any uniform error of weight r . The DFR that the AG codes decode $\mathbf{e}$ is less than or equal to $\gamma_q \cdot q^{a(t+r-a-n)}$, where $a = t-k-r+1$, and $\gamma_q = \prod_{j=1}^{\infty} \frac{1}{1-q^{-j}}$ is always greater than 1 and is monotonically decreasing in q , e.g., $\gamma_2 \approx 3.463$, $\gamma_3 \approx 1.785$, and $\gamma_4 \approx 1.452$.*

3.3 Public Key Encryption (C-Multi-UR-AG.PKE)

C-Multi-UR-AG.PKE uses three codes:

- A public $[N_1N_2, k]_{q^m}$ -AG code with generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times N_1N_2}$ generated by $\mathbf{g} \in \mathbb{F}_{q^m}^{N_1N_2}$ of weight t . This AG code can correct up to weight $\min \left\{ \left\lfloor \frac{N_1N_2-k}{2} \right\rfloor, t-k \right\}$ by a probabilistic decoding algorithm denoted by AG.Decode.
- A random $[2n, n]_{q^m}$ -linear code with parity-check matrix $[\mathbf{I}_n \quad \mathbf{H}]$.
- A random $[2n + N_1, n]_{q^m}$ -linear code with parity-check matrix $\begin{bmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{H}^\top \\ \mathbf{0} & \mathbf{I}_{N_1} & \mathbf{S}^\top \end{bmatrix}$.

The AG code is used for the decryption step. Two random linear codes are used to ensure the security of the scheme. C-Multi-UR-AG.PKE is described in Algorithms 2–5.

C-Multi-UR-AG.PKE uses two operations:

- **Fold**: It takes as input a vector of length N_1N_2 and outputs a matrix of size $N_1 \times N_2$ by a specific rule.
- **UnFold**: The inverse operation of **Fold**. It takes as input a matrix of size $N_1 \times N_2$ and output a vector of length N_1N_2 by the specific rule as in the operation **Fold**.

C-Multi-UR-AG.PKE uses two sets of small weight matrices:

- $\mathcal{S}_{w_1,1}^{2n \times N_1} = \left\{ \mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \end{bmatrix} : \mathbf{X}_1 \in \mathbb{F}_{q^m}^{n \times N_1}, \mathbf{X}_2 \in \mathbb{F}_{q^m}^{n \times N_1}, \|\mathbf{X}\|_R = w_1, 1 \in \text{Supp}(\mathbf{X}) \right\}$.
- $\mathcal{S}_{w_2}^{(2n+N_1) \times N_2} = \left\{ \mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \mathbf{X}_3 \end{bmatrix} : \mathbf{X}_1 \in \mathbb{F}_{q^m}^{n \times N_2}, \mathbf{X}_2 \in \mathbb{F}_{q^m}^{N_1 \times N_2}, \mathbf{X}_3 \in \mathbb{F}_{q^m}^{n \times N_2}, \|\mathbf{X}\|_R = w_2 \right\}$.

Correctness. In the decryption step,

$$\text{UnFold}(\mathbf{V} - \mathbf{Y}^\top \mathbf{U}) = \mathbf{m}\mathbf{G} + \text{UnFold}(\mathbf{X}^\top \mathbf{R}_2 - \mathbf{Y}^\top \mathbf{R}_1 + \mathbf{E}).$$

The term $\text{UnFold}(\mathbf{X}^\top \mathbf{R}_2 - \mathbf{Y}^\top \mathbf{R}_1 + \mathbf{E})$ is viewed as an error of length N_1N_2 and

$$\left\| \text{UnFold}(\mathbf{X}^\top \mathbf{R}_2 - \mathbf{Y}^\top \mathbf{R}_1 + \mathbf{E}) \right\|_R \leq w_1w_2.$$

Let $r = w_1w_2$. To obtain $\mathbf{m}$, one applies AG.Decode to $\text{UnFold}(\mathbf{V} - \mathbf{Y}^\top \mathbf{U})$. Thus, one must ensure that the weight $r \leq \min \left\{ \left\lfloor \frac{N_1N_2-k}{2} \right\rfloor, t-k \right\}$ and $\mathbf{m}$ can be decoded by AG.Decode. Moreover, one also ensures that the decoding failure rate (DFR) (see Theorem 3.1) is small enough for correctness. This DFR determines the decryption failure rate δ of C-Multi-UR-AG.PKE. Let δ be equal to DFR, then C-Multi-UR-AG.PKE is $(1 - \text{DFR})$ -correct. To avoid abuse of notation, we use DFR to denote decryption (or decoding) failure rate.

Security. In Section 6, we show that C-Multi-UR-AG.PKE is tightly IND-CPA secure under the RSL hardness assumption. This security is independent of DFR. The DFR solely guarantees the correctness of decryption. In other words, the DFR does not affect IND-CPA security; it only influences the probability that a honestly generated ciphertext decrypts correctly. However, there is an inherent trade-off: for an fixed security strength, achieving a smaller DFR comes at the cost of larger parameters.

Algorithm 2 C-Multi-UR-AG.PKE.Setup(λ)

Input: the security level parameter λ

Output: the global system parameters par

- 1: Set the global system parameters $\text{par} = (q, m, n, k, t, w_1, w_2, N_1, N_2)$
-

Algorithm 3 C-Multi-UR-AG.PKE.KGen(par)

Input: the global system parameters par

Output: a public key $\text{pk} := (\mathbf{g}, \mathbf{H}, \mathbf{S})$ and a private key $\text{sk} := (\mathbf{X}, \mathbf{Y})$

- 1: Sample $\mathbf{g} \xleftarrow{\theta_1} \mathcal{S}_t^{N_1 N_2}$ from a seed θ_1
 - 2: Sample $\mathbf{H} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^{n \times n}$ from a seed θ_1
 - 3: Sample $\begin{bmatrix} \mathbf{X} \\ \mathbf{Y} \end{bmatrix} \xleftarrow{\theta_2} \mathcal{S}_{w_1, 1}^{2n \times N_1}$ from a seed θ_2
 - 4: Compute $\mathbf{S} = \mathbf{X} + \mathbf{H}\mathbf{Y} \in \mathbb{F}_{q^m}^{n \times N_1}$
-

Algorithm 4 C-Multi-UR-AG.PKE.Enc($\text{pk}, \mathbf{m}; \theta$)

Input: the public key pk , a plaintext $\mathbf{m} \in \mathbb{F}_{q^m}^k$, and a seed θ

Output: a ciphertext $\text{ct} := (\mathbf{U}, \mathbf{V})$

- 1: Compute the generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times N_1 N_2}$ of $[N_1 N_2, k]_{q^m}$ -AG code defined by $\mathbf{g}$
 - 2: Sample $\begin{bmatrix} \mathbf{R}_1 \\ \mathbf{E} \\ \mathbf{R}_2 \end{bmatrix} \xleftarrow{\theta} \mathcal{S}_{w_2}^{(2n+N_1) \times N_2}$
 - 3: Compute $\mathbf{U} = \mathbf{R}_1 + \mathbf{H}^\top \mathbf{R}_2 \in \mathbb{F}_{q^m}^{n \times N_2}$ and $\mathbf{V} = \text{Fold}(\mathbf{m}\mathbf{G}) + \mathbf{E} + \mathbf{S}^\top \mathbf{R}_2 \in \mathbb{F}_{q^m}^{N_1 \times N_2}$
-

Algorithm 5 C-Multi-UR-AG.PKE.Dec(sk, ct)

Input: the private key sk and the ciphertext ct

Output: the plaintext $\mathbf{m}$

- 1: Compute $\mathbf{V} - \mathbf{Y}^\top \mathbf{U} = \text{Fold}(\mathbf{m}\mathbf{G}) + \mathbf{X}^\top \mathbf{R}_2 - \mathbf{Y}^\top \mathbf{R}_1 + \mathbf{E}$
 - 2: Recover the plaintext $\mathbf{m} := \text{AG.Decode}(\text{UnFold}(\mathbf{V} - \mathbf{Y}^\top \mathbf{U}))$
-

3.4 Key Encapsulation Mechanism (C-Multi-UR-AG.KEM)

In this section, we consider the KEM version of C-Multi-UR-AG (C-Multi-UR-AG.KEM), an IND-CCA2 secure KEM, from C-Multi-UR-AG.PKE described in Algorithms 2–5 by the salted FO transformation [30]. Let $\text{PKE} := \text{C-Multi-UR-AG.PKE}$ and $\text{KEM} := \text{C-Multi-UR-AG.KEM}$. The resulting KEM is described in Algorithms 6–9.

Correctness. The correctness follows PKE. Assume that PKE is $(1 - \delta)$ -correct. As KEM only includes one PKE ciphertext, KEM is also $(1 - \delta)$ -correct, i.e., the decapsulation failure rate is δ . Moreover, one ensures that δ is small enough for correctness.

Security. KEM is IND-CCA2 secure. In Section 6 (Theorem 6.2), this security is tight with respect to the hardness of the RSL assumption and is related to δ . Unlike CPA security of PKE, CCA security is sensitive to δ : a lower DFR directly enhances the tightness and reliability of the CCA reduction. Thus, the DFR of PKE must be set to be small enough for CCA security of KEM. We set $\text{DFR} \leq 2^{-\lambda}$ for the target security level parameter λ .

Algorithm 6 KEM.Setup(λ)

Input: the security level parameter λ

Output: the global system parameters par

- 1: Set the global system parameters $\text{par} \leftarrow \text{PKE.Setup}(\lambda)$
-

Algorithm 7 KEM.KGen(par)

Input: the global system parameters par

Output: a public key pk and a private key sk

- 1: $(\text{pk}', \text{sk}') \leftarrow \text{PKE.KGen}(\text{par})$
 - 2: Sample $\sigma \xleftarrow{\$} \mathbb{F}_{q^m}^k$
 - 3: Set the public key $\text{pk} := \text{pk}'$ and the private key $\text{sk} := (\text{sk}', \sigma)$
-

Algorithm 8 KEM.Encaps(pk)

Input: the public key pk

Output: a ciphertext and key pair (ct, K)

- 1: Sample $\mathbf{m} \xleftarrow{\$} \mathbb{F}_{q^m}^k$ and $\text{salt} \xleftarrow{\$} \{0, 1\}^{\text{len}}$
 - 2: Compute $\theta = G(\text{pk}, \mathbf{m}, \text{salt})$
 - 3: $\text{ct}' \leftarrow \text{PKE.Enc}(\text{pk}, \mathbf{m}; \theta)$
 - 4: $\text{ct} := (\text{ct}', \text{salt})$
 - 5: Compute $K = H(\text{pk}, \mathbf{m}, \text{ct}', \text{salt})$
-

Algorithm 9 KEM.Decaps(sk, ct)

Input: the private key sk and the ciphertext ct

Output: the key K

- 1: Parse $\text{sk} := (\text{sk}', \sigma)$ and $\text{ct} := (\text{ct}', \text{salt})$
 - 2: $\overline{\mathbf{m}} \leftarrow \text{PKE.Dec}(\text{sk}', \text{ct}')$ and compute $\overline{\theta} = G(\text{pk}, \overline{\mathbf{m}}, \text{salt})$
 - 3: $\overline{\text{ct}} \leftarrow \text{PKE.Enc}(\text{pk}, \overline{\mathbf{m}}; \overline{\theta})$
 - 4: If $\overline{\text{ct}} \neq \text{ct}'$, set $K := H(\text{pk}, \sigma, \overline{\text{ct}}, \text{salt})$; else, set $K := H(\text{pk}, \overline{\mathbf{m}}, \overline{\text{ct}}, \text{salt})$
-

4 Parameters

In this section, we give concrete parameter sets. The proposed parameter sets cover security levels: **128**-, **256**-, and **512**-bit security.

Each parameter set is chosen according to four principles.

- From Theorems 6.1 and 6.2, the security of schemes is tightly reduced to the hardness of two RSL assumptions. Thus, the minimal complexity of known attacks against two RSL assumptions reaches the security level 2^λ for the target security level parameter λ . As analyzed in [8, 16, 22, 25], the RSL problem is almost as hard as the RSD problem for a small number of syndromes N_1 and N_2 (see Section 2.3 for details). Thus, we use the complexity of the RSD problem to describe that of the RSL problem. The main attacks against the RD problem are recalled in Section 7.
- The decoding capacity of the AG codes is ensured to satisfy the decryption correctness condition, namely decoding weight up to r .
- We require that the DFR correcting errors of weight up to r (Theorem 3.1 for DFR) is less than or equal to $2^{-\lambda}$ for the target security level parameter λ .

- We set a sufficiently large plaintext space (i.e., $\mathbb{F}_{q^m}^k$). We assume that the number of queries to the hash oracles is q^{64} . In the classical model, by Theorem 6.2, we require that the size of the plaintext space (i.e., pts in Table 3, q^{mk}) at least satisfies: $|\mathbb{F}_{q^m}^k|/2^{64} \geq 2^\lambda$. Considering underlying quantum attacks, we set $\sqrt{|\mathbb{F}_{q^m}^k|/2^{64}} \geq 2^\eta$ where η is the required quantum security levels: 80, 128, and 256.

In Table 2, we select the security parameters for three security levels. In Table 3, we compute sizes and DFR, and estimate classical/quantum security strength and space complexity.

Table 2: Parameter Sets

Instance	q	m	n	t	k	N_1	N_2	w_1	w_2	r
Level-128	2	79	35	79	3	11	16	8	9	72
Level-256	2	127	45	127	3	15	16	10	11	110
Level-512	2	181	67	181	4	19	21	12	14	168

Table 3: Sizes, DFR, classical/quantum security strength, and space complexity

Instance	pts	sks	pks	cts	total	$ K $	DFR	Classical Security Strength	Quantum Security Strength	Space Complexity
Level-128	237	94	3866	7332	11198	64	2^{-148}	2^{170}	2^{85}	2^{71}
Level-256	381	112	10780	15304	26084	64	2^{-268}	2^{276}	2^{138}	2^{89}
Level-512	724	155	28866	40926	69792	64	2^{-598}	2^{530}	2^{265}	2^{98}

pts: the plaintext size, mk bits. sks: the private key size, $\lceil \frac{mk}{8} \rceil + 64$ bytes. pks: the public key size, $\lceil \frac{mnN_1}{8} \rceil + 64$ bytes. cts: the ciphertext size, $\lceil \frac{N_2nm}{8} \rceil + \lceil \frac{N_1N_2m}{8} \rceil + 64$ bytes. total = pks + cts. $|K|$: the shared key size, 64 bytes. DFR: decapsulation failure rate. The DFR is computed by Theorem 3.1 with $\gamma_q = 4$. The classical security strengths are determined by the MM attack (see Table 6). The quantum security strengths are the square root of the classical security strengths. The space complexity corresponds to the classical security strength.

5 Performance Analysis

This section provides performance measures of C-Multi-UR-AG.KEM.

Benchmark platform. All benchmarks were conducted inside a WSL 2 environment running Ubuntu 26.04 LTS on a Windows laptop equipped with an Intel[®] Core[™] Ultra 9 275HX processor (8 performance cores, base frequency 2.70 GHz, with turbo frequencies up to 5.40 GHz, 16 efficient cores, 24 total threads) and 64 GB of memory. The software implementation environment is configured with support for the following instruction sets: MMX, SSE (including SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2), EM64T (Intel 64), AES, AVX, AVX2, AVX-VNNI, FMA3, and SHA. To avoid performance fluctuations caused by migration across the heterogeneous core types, each benchmark process was pinned to a single performance core (CPU 0, a P-core) via `sched_setaffinity`.

The implementation was compiled with gcc version 15.2.0 and relies on the RBC library (version 1.3) [4]. For each parameter set, the reported value is the mean over 50 random key instances; each optimized instance is timed over 100 inner iterations and each reference instance over 10 inner iterations. Since WSL 2 does not expose CPU frequency-scaling controls, Turbo

Boost remains active and introduces rare slow samples that inflate the arithmetic mean of short-running operations. For the optimized implementation, we therefore report the median which is robust to such residual frequency jitter. The reference implementation runs long enough per sample that its distribution is unimodal, and its mean is reported directly.

Constant time. The provided implementations are conducted in a constant time way, so that their running time does not leak any information with respect to sensitive data such as the secret key, the weight of the error to be decoded by the AG code, or the validity of the received ciphertext. In particular, the Fujisaki–Okamoto re-encryption check in **Decaps** compares (U, V) and (U', V') byte-by-byte with an OR-accumulator, and selects between the decoded message m and the implicit-rejection value σ using a constant-time bitmask instead of a branch.

5.1 Reference Implementation

The performance of our reference implementation on the aforementioned benchmark platform is described in Table 4. The reference implementation was compiled with `-std=c99 -Wpedantic -Wall -Wextra -O2`. Values are mean CPU cycles in kiloCycles.

Table 4: Performance of C-Multi-UR-AG reference implementation (in kiloCycles)

Instance	KGen	Encaps	Decaps
Level-128	7 761.8	14 388.1	78 328.8
Level-256	23 252.7	33 425.5	206 285.7
Level-512	162 097.1	234 315.2	1 440 824.1

5.2 Optimized Implementation

An x86 performance-optimized implementation is provided. Its performances on the aforementioned benchmark platform are described in Table 5. The optimized implementation was compiled with `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -mpclmul -msse4.2 -maes -std=c99 -Wpedantic -Wall -Wextra -DSHAKE_TIMES4`. Values are median CPU cycles in kiloCycles.

Table 5: Performance of C-Multi-UR-AG optimized implementation (in kiloCycles)

Instance	KGen	Encaps	Decaps
Level-128	845.2	1 145.3	2 922.9
Level-256	2 017.4	2 483.4	6 510.6
Level-512	9 155.2	11 476.5	35 660.6

6 Security Proof

In this section, we first prove that PKE is a tight IND-CPA secure under the RSL assumption, then obtain that KEM is a tight IND-CCA2 secure under the same hardness assumption due to the FO transformation.

Theorem 6.1. *Let $\text{PKE} := \text{C-Multi-UR-AG.PKE}$ as Algorithms 2–5. Under the $\text{DRSL}_{q,m,2n,n,w_1,N_1}$ and $\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}$ problems, PKE is IND-CPA secure. Concretely, for any PPT algorithm $\mathcal{A}$, there exists a PPT algorithm $\mathcal{B}$ such that the advantage that $\mathcal{A}$ against the IND-CPA experiment defined by PKE is bounded as*

$$\text{Adv}_{\text{PKE},\mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n,n,w_1,N_1}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}}(\lambda).$$

Proof. We prove the theorem using a sequence of games $\mathbf{G}_i$ for $i \in \{0, 1, 2\}$. We define

$$\text{Adv}_i := \Pr[\mathbf{G}_i \Rightarrow 1], \quad \text{for } i \in \{0, 1, 2\}.$$

Let $\mathbf{G}_0$ be the real IND-CPA game (see Figure 1) and define $\mathbf{G}_0 := \text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda)$. We introduce changes to $\mathbf{G}_0$ and end up at $\mathbf{G}_2$. We argue that $\mathbf{G}_2$ outputs 1 with a probability $1/2$. The games are presented in Figure 2.

Game $\mathbf{G}_0$: $\mathbf{G}_0$ works as follows. First, it samples $\text{par} \leftarrow \text{Setup}(1^\lambda)$. Then, it computes the public key and the private key: sampling $\mathbf{g} \xleftarrow{\theta_1} \mathcal{S}_t^{N_1 N_2}$ and $\mathbf{H} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^{n \times n}$, sampling $(\mathbf{X}, \mathbf{Y}) \xleftarrow{\theta_2} \mathcal{S}_{w_1,1}^{2n \times N_1}$, and computing $\mathbf{S} = \mathbf{X} + \mathbf{H}\mathbf{Y}$. The public key $\text{pk} := (\mathbf{g}, \mathbf{H}, \mathbf{S})$ and the private key $\text{sk} := (\mathbf{X}, \mathbf{Y})$. Next, $\mathcal{A}$ is given pk and par and samples two plaintexts $\mathbf{m}_0, \mathbf{m}_1$ with the equal length. The game samples $b \xleftarrow{\$} \{0, 1\}$, runs the encryption algorithm PKE.Enc , and computes the ciphertext $\text{ct} \leftarrow \text{PKE.Enc}(\text{pk}, \mathbf{m}_b, \theta)$ (see Line 08 in Figure 2). Finally, $\mathcal{A}$ outputs $b' \in \{0, 1\}$ with ct . The game outputs 1, if $b' = b$. By definition, we have

$$\text{Adv}_0 = \Pr[\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda) \Rightarrow 1].$$

Game $\mathbf{G}_1$: We change $\mathbf{G}_0$ into $\mathbf{G}_1$. Recall that $\mathbf{S} = \begin{bmatrix} \mathbf{I}_n & \mathbf{H} \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ \mathbf{Y} \end{bmatrix}$. In this game, we sample $\mathbf{S} \xleftarrow{\$} \mathbb{F}_{q^m}^{n \times N_1}$. We argue that two games are computationally close by the DRSL assumption. Thus, we have

$$|\text{Adv}_0 - \text{Adv}_1| \leq \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n,n,w_1,N_1}}(\lambda).$$

Game $\mathbf{G}_2$: We change $\mathbf{G}_1$ into $\mathbf{G}_2$. Recall that $\begin{bmatrix} \mathbf{U} \\ \mathbf{V} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{H}^\top \\ \mathbf{0} & \mathbf{I}_{N_1} & \mathbf{S}^\top \end{bmatrix} \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{E} \\ \mathbf{R}_2 \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \text{Fold}(\mathbf{m}_b \mathbf{G}) \end{bmatrix}$.

In this game, we sample $(\mathbf{U}, \mathbf{V}) \xleftarrow{\$} \mathbb{F}_{q^m}^{n \times N_2} \times \mathbb{F}_{q^m}^{N_1 \times N_2}$ and replace $\begin{bmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{H}^\top \\ \mathbf{0} & \mathbf{I}_{N_1} & \mathbf{S}^\top \end{bmatrix} \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{E} \\ \mathbf{R}_2 \end{bmatrix}$ by $\begin{bmatrix} \mathbf{U} \\ \mathbf{V} - \text{Fold}(\mathbf{m}_b \mathbf{G}) \end{bmatrix}$. We can observe that $\mathbf{V} - \text{Fold}(\mathbf{m}_b \mathbf{G})$ is uniformly random due to the uniform randomness of $\mathbf{v}$. We argue that two games are computationally close by the DRSL assumption. Thus, we have

$$|\text{Adv}_1 - \text{Adv}_2| \leq \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}}(\lambda).$$

At this point, in the game $\mathbf{G}_2$, all information $(\text{par}, \text{pk}, \text{ct})$ given to $\mathcal{A}$ is uniformly random and independent of the private key and the randomness. Thus,

$$\text{Adv}_2 = \Pr[\mathbf{G}_2 \Rightarrow 1] = \frac{1}{2}.$$

Combining equations obtained in games $\mathbf{G}_0$ - $\mathbf{G}_2$, we have

$$|\text{Adv}_0 - \text{Adv}_2| \leq \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n,n,w_1,N_1}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}}(\lambda).$$

Game G_0-G_2	
01 $\text{par} \leftarrow \text{Setup}(1^\lambda)$	
02 $\mathbf{g} \xleftarrow{\theta_1} \mathcal{S}_t^{N_1 N_2}, \mathbf{H} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^{n \times n}$	
03 $\begin{bmatrix} \mathbf{X} \\ \mathbf{Y} \end{bmatrix} \xleftarrow{\theta_2} \mathcal{S}_{w_1,1}^{2n \times N_1}, \mathbf{S} = [\mathbf{I}_n \quad \mathbf{H}] \begin{bmatrix} \mathbf{X} \\ \mathbf{Y} \end{bmatrix} \in \mathbb{F}_{q^m}^{n \times N_1}$	
04 $\mathbf{S} \xleftarrow{\$} \mathbb{F}_{q^m}^{n \times N_1}$	// G_1
05 $\text{pk} := (\mathbf{g}, \mathbf{H}, \mathbf{S}), \text{sk} := (\mathbf{X}, \mathbf{Y})$	
06 $(\mathbf{m}_0, \mathbf{m}_1) \xleftarrow{\$} \mathbb{F}_{q^m}^k \times \mathbb{F}_{q^m}^k$	
07 $b \xleftarrow{\$} \{0, 1\}$	
08 $\begin{bmatrix} \mathbf{R}_1 \\ \mathbf{E} \\ \mathbf{R}_2 \end{bmatrix} \xleftarrow{\theta_3} \mathcal{S}_{w_2}^{(2n+N_1) \times N_2}, \begin{bmatrix} \mathbf{U} \\ \mathbf{V} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_n & \mathbf{0} & \mathbf{H}^\top \\ \mathbf{0} & \mathbf{I}_{N_1} & \mathbf{S}^\top \end{bmatrix} \begin{bmatrix} \mathbf{R}_1 \\ \mathbf{E} \\ \mathbf{R}_2 \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \text{Fold}(\mathbf{m}_b \mathbf{G}) \end{bmatrix}$	
09 $(\mathbf{U}, \mathbf{V}) \xleftarrow{\$} \mathbb{F}_{q^m}^{n \times N_2} \times \mathbb{F}_{q^m}^{N_1 \times N_2}, \begin{bmatrix} \mathbf{U} \\ \mathbf{V} \end{bmatrix} = \begin{bmatrix} \mathbf{U} \\ \mathbf{V} - \text{Fold}(\mathbf{m}_b \mathbf{G}) \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \text{Fold}(\mathbf{m}_b \mathbf{G}) \end{bmatrix}$	// G_2
10 $\text{ct} = (\mathbf{U}, \mathbf{V})$	
11 $b' \leftarrow \mathcal{A}(\text{par}, \text{pk}, \text{ct})$	
12 $b' \neq b : \text{return } 0$	
13 return 1	

Figure 2: The games G_0 - G_2 in the proof of Theorem 6.1. Annotated lines using “//” are only executed in the corresponding games.

$\Longleftrightarrow$

$$\left| \Pr [\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n,n,w_1,N_1}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}}(\lambda).$$

$\Longleftrightarrow$

$$\text{Adv}_{\text{PKE},\mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n,n,w_1,N_1}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}}(\lambda).$$

□

C-Multi-UR-AG.KEM Security. KEM is obtained by applying the salted FO transformation [30] to C-Multi-UR-AG.PKE. Following the salted FO transformation, an IND-CPA secure PKE tightly implies an IND-CCA2 secure KEM, our KEM is IND-CCA2 secure. When the hash functions G and H are modeled as random oracles, by combining the generic bounds (Theorems 1 and 3 in [30]), we have Theorem 6.2.

Theorem 6.2. *Let $\mathcal{M}$ be the plaintext space of PKE with a DFR δ . Assume that G and H are random oracles. For any classical adversary $\mathcal{A}'$ that makes at most q_G queries to G , q_H queries*

to H , and q_D queries to the decapsulation oracle, there exists an adversary $\mathcal{A}$ of roughly the same running time as that of $\mathcal{A}'$ such that

$$\text{Adv}_{\text{KEM}, \mathcal{A}'}^{\text{IND-CCA2}}(\lambda) \leq 2\text{Adv}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) + \frac{1}{|\mathcal{M}| \cdot 2^{\text{len}}} + \frac{2q_H + q_G}{|\mathcal{M}|} + (q_H + q_D)\delta.$$

Combining Theorems 6.1 and 6.2, we have that the IND-CCA2 security of KEM is tight with respect to the hardness of the $\text{DRSL}_{q,m,2n,n,w_1,N_1}$ and $\text{DRSL}_{q,m,2n+N_1,n,w_2,N_2}$ assumptions.

7 Known Attacks on the RSD Problem

Theorem 7.1. *Solving the RD problem defined by an $[n, k]_{q^m}$ linear code $\mathcal{C}$ and instantiated by (G, y, e) is equivalent to finding a codeword belonging to $\mathcal{S}_r^n$ in the $[n, k+1]_{q^m}$ extended code $\mathcal{C}_y$ of $\mathcal{C}$.*

Proof. Once obtaining word y , one adds y to $\mathcal{C}$ and obtains an $[n, k+1]_{q^m}$ extended code $\mathcal{C}_y = \mathcal{C} + \langle y \rangle$ with a generator matrix $\begin{bmatrix} y \\ G \end{bmatrix}$ of size $(k+1) \times n$. In this way, $e = (1 \ -m) \begin{bmatrix} y \\ G \end{bmatrix}$ is exactly a codeword of $\mathcal{C}_y$. Let $G_y \in \mathbb{F}_{q^m}^{(k+1) \times n}$ be a generator matrix of $\mathcal{C}_y$ and $H_y \in \mathbb{F}_{q^m}^{(n-k-1) \times n}$ be a parity-check matrix of $\mathcal{C}_y$. Then solving the RD problem is equivalent to finding an $u \in \mathbb{F}_{q^m}^{k+1}$ such that

$$uG_y = e, \tag{1}$$

or finding an error $e \in \mathcal{S}_r^n$ such that

$$eH_y^\top = 0_{n-k-1}. \tag{2}$$

□

The Structure of Standard Errors. Recalled that any random standard error $e \in \mathcal{S}_r^n$ can be expressed as $e = \varepsilon C = \alpha SC$ where $\varepsilon \in \mathbb{F}_q^r$ is e 's basis, $S \in \mathbb{F}_q^{m \times r}$ is e 's support matrix, and $C \in \mathbb{F}_q^{r \times n}$ is e 's coefficient matrix. S (or C) can be reduced to a systematic form, namely the first r rows (or the first r columns) form an identity matrix I_r .

7.1 Combinatorial Attacks

The typical combinatorial attacks on the RD problem have been considered in [5, 21, 39, 40]. The powerful attacks are the AGHT attack and the OJ attack.

The AGHT Attack. The AGHT attack [6] is proposed by Aragon, Gaborit, Hauteville, and Tillich for solving the RD problem and is an improvement of GRS combinatorial attack [26]. The attack is divided into two strategies. Here we call **AGHT-S** and **AGHT-C**, respectively. They guess a subspace that respectively contains the column space of S and the row space of C , and check if the guess is correct by solving the linear system obtained from the parity-check equations. The former is the case of cryptographic parameters in this submission.

AGHT-S: The solver guesses subspace that contains the column space of S .

- Let E be the r -dimensional support of e . It guesses a t -dimensional $\mathbb{F}_q$ -subspace F such that F contains E . Equivalently, it guesses randomly a t -dimensional subspace T of $\mathbb{F}_q^m$ such that T contains the r -dimensional column space of S . Let $T \in \mathbb{F}_q^{m \times t}$ be the basis

matrix of T . Let $\mathbf{f} = \boldsymbol{\alpha}T \in \mathbb{F}_{q^m}^t$. In this way, the $\mathbb{F}_q$ -subspace spanned by $\mathbf{f}$ is t -dimensional and contains E . We denote such a t -dimensional $\mathbb{F}_q$ -subspace as F , whose basis is $\mathbf{f}$. Due to $F \supset E$, $\mathbf{e}$ can be expressed by the basis $\mathbf{f}$ of F . Then there exists a matrix $\mathbf{F} \in \mathbb{F}_q^{t \times n}$ such that $\mathbf{e} = \mathbf{f}\mathbf{F}$.

- Express the blockwise error $\mathbf{e}$ as $\mathbf{e} = \mathbf{f}\mathbf{F} \in \mathbb{F}_{q^m}^n$.
- From the parity-check Equation (2): $\mathbf{e}\mathbf{H}_y^\top = \mathbf{0}$, construct a linear system $\mathbf{f}\mathbf{F}\mathbf{H}_y^\top = \mathbf{0}$ and unfold it over $\mathbb{F}_q$ to obtain a linear system with $\sum_{i=1}^\ell n_i t_i$ unknowns in the entries of $\mathbf{F}$ and $m(n-k-1)$ equations over $\mathbb{F}_q$.
- Solve the obtained linear system for a single solution as long as $nt \leq m(n-k-1)$. (If the system could be solved, then the guess is correct.)
- Compute $\mathbf{f}\mathbf{F}$ as $\mathbf{e}$ if the entries of $\mathbf{F}$ are solved.

The most costly part of the attack consists in successfully finding F containing E . This probability is estimated as $q^{r(m-t)}$.

Then one uses $\mathbb{F}_{q^m}$ -linearity to improve the success probability and reduce the cost. Since, for almost any $\beta \in \mathbb{F}_{q^m}^*$, $\|\beta\mathbf{e}\|_R = r$, $\text{Supp}(\beta\mathbf{e}) = \beta\text{Supp}(\mathbf{e}) = \beta E$, and all multiples $\beta\mathbf{e}$ are solutions of Equation (2): $\mathbf{e}\mathbf{H}_y^\top = \mathbf{0}$, one can find the solution of form $\beta\mathbf{e}$. Then one can guess T such that the t -dimensional $\mathbb{F}_q$ -subspace F spanned by $\mathbf{f}$ contains the r -dimensional support of the form βE . For the fixed E , the number of βE is at most $\frac{q^m-1}{q-1}$. The probability of $F \supset \beta E$ is estimated as $\frac{q^m-1}{q-1} q^{-r(m-t)} \approx q^{-r(m-t)+m}$. As a result, this strategy has a complexity of $\mathcal{O}((m(n-k-1))^\omega q^{r(m-t)-m})$ over $\mathbb{F}_q$.

AGHT-C: The solver guesses a subspace that contains the row space of $\mathbf{C}$.

- Guess randomly t -dimensional subspace F' of $\mathbb{F}_q^n$ such that F' contains the r -dimensional row space C of $\mathbf{C}$. Let the matrix $\mathbf{F}' \in \mathbb{F}_q^{t' \times n}$ be a basis matrix of F' . Then there exists $\mathbf{f}' \in \mathbb{F}_{q^m}^{t'}$ such that $\mathbf{e} = \mathbf{f}'\mathbf{F}'$.
- Express the blockwise error $\mathbf{e}$ as $\mathbf{e} = \mathbf{f}'\mathbf{F}' \in \mathbb{F}_{q^m}^n$.
- From the parity-check equation: $\mathbf{e}\mathbf{H}^\top = \mathbf{s}$, construct a linear system $\mathbf{f}'\mathbf{F}'\mathbf{H}^\top = \mathbf{s}$ with t' unknowns in the entries of $\mathbf{f}'$ and $n-k$ equations over $\mathbb{F}_{q^m}$.
- Solve the obtained linear system for single solution $\mathbf{f}'$ as long as $t' \leq n-k$. (If the system could be solved, then the guess is correct.)
- Compute $\mathbf{f}'\mathbf{F}'$ as $\mathbf{e}$ if $\mathbf{f}'$ is solved.

The most costly part of the attack consists in successfully finding F' containing C . This probability is estimated as $q^{-r(n-t')}$. The fourth step costs $\mathcal{O}((n-k)^\omega m^2)$ operations over $\mathbb{F}_q$. As a result, this strategy has a complexity of $\mathcal{O}((n-k)^\omega m^2 q^{rk})$ over $\mathbb{F}_q$.

Theorem 7.2. *The time complexity of solving the RD problem by the AGHT attack is estimated as*

$$\min \left(((n-k-1)m)^\omega q^{r \lceil \frac{(k+1)m}{n} \rceil - m}, (n-k)^\omega m^2 q^{rk} \right)$$

over $\mathbb{F}_q$ where ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$.

Recently, the work [21] proposed an enhanced AGHT-S attack. In this attack, the adversary guesses entries of the matrix $\mathbf{Mat}(\mathbf{e}) \in \mathbb{F}_q^{n \times n}$, where each guess yields an additional linear equation in the entries of $\mathbf{F}$. The cost of making η such guesses is q^η . As a result, one obtains a total of $(n - k - 1)m + \eta$ linear equations in the nt unknown entries of $\mathbf{F}$. The complexity is given in Theorem 7.3. In fact, for our parameters, the best AGHT attack is AGHT-S.

Theorem 7.3 (Enhanced AGHT-S Attack). *The time complexity of solving the RD problem over $\mathbb{F}_{q^m}$ by the enhanced AGHT-S attack is estimated as*

$$\mathcal{O} \left(((n - k - 1)m + \eta)^\omega q^{r \left\lceil \frac{(k+1)m - \eta}{n} \right\rceil - m + \eta} \right),$$

over $\mathbb{F}_q$ where $\eta \leq nm$, and ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$.

The OJ Attack. The OJ attack [35] is proposed by Ourivski and Johansson for the RD problem. Recently, this attack is refined in [37], where explicit modeling and general complexity formula are given. The general idea of the OJ attack is that the solver constructs a quadratic multivariate system about the entries of matrices $\mathbf{S}$ and $\mathbf{C}$ from Equation (1): $\mathbf{uG}_y = \mathbf{e}$, then guesses the entries of $\mathbf{S}$ (resp. $\mathbf{C}$) to transform the quadratic multivariate system into a linear system about the entries of $\mathbf{C}$ (resp. $\mathbf{S}$). The cost depends on the probability of successfully guessing the entries of the matrices.

We start with Theorem 7.1. Let $\mathbf{G}_y = [\mathbf{I}_{k+1} \ \mathbf{R}] \in \mathbb{F}_{q^m}^{(k+1) \times n}$ be a systematic generator matrix of $\mathcal{C}_y$ where $\mathbf{R} \in \mathbb{F}_{q^m}^{(k+1) \times (n-k-1)}$. Let $\bar{\mathbf{e}}_1$ and $\bar{\mathbf{e}}_2$ be the first $k+1$ and the last $n-k-1$ coordinates of $\mathbf{e}$, respectively. Let $\mathbf{A}_1$ and $\mathbf{A}_2$ be the first $k+1$ columns and the last $n-k-1$ columns of $\mathbf{C}$, respectively. Then

$$\begin{aligned} \mathbf{e} = (\bar{\mathbf{e}}_1, \bar{\mathbf{e}}_2) &= \alpha \mathbf{S} \mathbf{C} = \alpha \mathbf{S} [\mathbf{A}_1, \mathbf{A}_2] = (\alpha \mathbf{S} \mathbf{A}_1, \alpha \mathbf{S} \mathbf{A}_2) \\ &\iff \\ \bar{\mathbf{e}}_1 &= \alpha \mathbf{S} \mathbf{A}_1, \quad \bar{\mathbf{e}}_2 = \alpha \mathbf{S} \mathbf{A}_2. \end{aligned} \tag{3}$$

From Equation (1), we have

$$\mathbf{uG}_y = \mathbf{e} \iff (\mathbf{u} \ \mathbf{uR}) = (\bar{\mathbf{e}}_1, \bar{\mathbf{e}}_2) \iff \bar{\mathbf{e}}_1 \mathbf{R} = \bar{\mathbf{e}}_2. \tag{4}$$

Combining Equation (3) and Equation (4), we have

$$\alpha \mathbf{S} \mathbf{A}_1 \mathbf{R} = \alpha \mathbf{S} \mathbf{A}_2. \tag{5}$$

By Equation (5), for $j \in \{1..n-k-1\}$, let $\mathbf{r}_j$ and $\mathbf{a}_j$ be the j -th column of $\mathbf{R}$ and $\mathbf{A}_2$, respectively, then

$$\alpha \mathbf{S} \mathbf{A}_1 \mathbf{r}_j = \alpha \mathbf{S} \mathbf{a}_j \iff \alpha \mathbf{S} [\mathbf{A}_1 \ \mathbf{a}_j] \begin{bmatrix} \mathbf{r}_j \\ -1 \end{bmatrix} = 0. \tag{6}$$

Let J be a fixed and random subset of $\{1..n-k-1\}$ and $\#J = N$. The OJ modeling is:

$$\left\{ P_j = \alpha \mathbf{S} [\mathbf{A}_1 \ \mathbf{a}_j] \begin{bmatrix} \mathbf{r}_j \\ -1 \end{bmatrix} : j \in J \right\} \tag{7}$$

Unknowns: $mr + (k+1+N)r$ entries of $\mathbf{S}$, $\mathbf{A}_1$, and $\mathbf{a}_j$ over $\mathbb{F}_q$,

Equations: N quadratic multivariate equations $P_j = 0$ over $\mathbb{F}_{q^m}$.

These equations are defined over $\mathbb{F}_{q^m}$, but the unknowns are in the basis field $\mathbb{F}_q$. Since $mr + (k + 1 + N)r \gg N$, the modeling is underdetermined and has many solutions. However, one wants more equations such that the modeling is overdetermined for a unique solution. To obtain more equations, one unfolds the coefficients of P_j over $\mathbb{F}_q$. The resulting modeling includes:

Unknowns: $mr + (k + 1 + N)r$ entries of $\mathbf{S}$, $\mathbf{A}_1$, and $\mathbf{a}_j$ over $\mathbb{F}_q$,

Equations: mN quadratic multivariate equations over $\mathbb{F}_q$.

Once the OJ modeling is solved, one can obtain the entries of $\mathbf{S}$. Then from Equation (2): $\mathbf{eH}_y^\top = \alpha \mathbf{S} \mathbf{C} \mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$, one can obtain $\mathbf{C}$. Finally, one determines $\mathbf{e}$ by computing $\alpha \mathbf{S} \mathbf{C}$.

The idea of solving the OJ modeling is that the solver transforms the quadratic multivariate system into a linear system with mN equations. The transformation includes the basis enumeration and the coordinate enumeration. The former guesses the entries of $\mathbf{S}$ to solve a linear system in the entries of $[\mathbf{A}_1 \ \mathbf{a}_j]$ to obtain $\mathbf{S}$. The latter guesses the entries of $[\mathbf{A}_1 \ \mathbf{a}_j]$ to solve a linear system in the entries of $\mathbf{S}$ to obtain $\mathbf{S}$. Thus, we here call **OJ-S** and **OJ-C**, respectively. For the coordinate enumeration, to obtain the whole $\mathbf{S}$, we must carefully choose $\mathbf{a}_j$ to construct a linear system whose unknowns include *all* entries of $\mathbf{S}$. The guess cost dominates the complexity of the OJ attack. A smaller number of entries in $\mathbf{S}$ (resp. $[\mathbf{A}_1 \ \mathbf{a}_j]$) leads to a more powerful basis enumeration (resp. coordinate enumeration). To obtain the best complexity, we must reduce support matrix $\mathbf{S}$ and coefficient matrix $\mathbf{C}$.

Reduced Support/Coefficient Matrices for Standard Errors We recall here the reduction of the standard errors. We use two well-known assumptions for any uniformly random standard errors:

- The first coordinate of $\mathbf{e}$ is 1. One can observe that Equation (2) has a large number of solutions. All multiples $\beta \mathbf{e}$ for any non-zero $\beta \in \mathbb{F}_{q^m}^*$ are its solutions because $\beta \mathbf{e}$ keeps the weight of $\mathbf{e}$. This shows that Equation (2) includes a solution whose first coordinate is 1 because β can be specified to the inverse of the first coordinate of $\mathbf{e}$. Let the first coordinate of the solution be 1, then such a solution is unique to Equation (2). We find such a solution. Under the basis α , one can set the first column of $\mathbf{C}$ to $(1 \ 0 \ \cdots \ 0)^\top$ and the first column of $\mathbf{S}$ to $(1 \ 0 \ \cdots \ 0)^\top$.
- The first r coordinates of $\mathbf{e}$ are linearly independent over $\mathbb{F}_q$. This probability is equal to the probability that the random $r \times r$ matrix over $\mathbb{F}_q$ is invertible, ≥ 0.289 for increased q . If it does not hold, we can perform an invertible matrix $\mathbf{P} \in \mathbb{F}_q^{n \times n}$ on the instance $(\mathbf{G}, \mathbf{y}, \mathbf{e})$ of the RD problem until the first r coordinates of $\mathbf{eP}$ are independent over $\mathbb{F}_q$. By the probability, such $\mathbf{P}$ is easy to be found. Then one solves a new instance $(\mathbf{GP}, \mathbf{yP}, \mathbf{eP})$. Solving these two instances is equivalent because $\mathbf{eP}$ keeps the support of $\mathbf{e}$.

Combining two assumptions, $\mathbf{S}$ and $\mathbf{C}$ can be further reduced to the following two forms with less entries.

- $\mathbf{S}_{\{1..r\},*} = \mathbf{I}_r$. By Gaussian elimination on columns of $\mathbf{S}$, there exists a matrix $\mathbf{P} = \left[\begin{array}{c|c} 1 & \mathbf{p} \\ \hline \mathbf{0}_{(r-1) \times 1} & \mathbf{P}' \end{array} \right] \in \text{GL}_r(\mathbb{F}_q)$ where $\mathbf{p} \in \mathbb{F}_q^{1 \times (r-1)}$ and $\mathbf{P}' \in \text{GL}_{r-1}(\mathbb{F}_q)$ such that $\mathbf{SP} = \left[\begin{array}{c|c} \mathbf{I}_r & \\ \hline \mathbf{0}_{(m-r) \times 1} & \mathbf{S}' \end{array} \right]$ and $\mathbf{P}^{-1}\mathbf{C} = \left[\begin{array}{c|c} 1 & \\ \hline \mathbf{0}_{(r-1) \times 1} & \mathbf{C}' \end{array} \right]$ where $\mathbf{S}' \in \mathbb{F}_q^{(m-r) \times (r-1)}$ and $\mathbf{C}' \in$

$\mathbb{F}_q^{r \times (n-1)}$. Then

$$\mathbf{e} = \alpha \mathbf{S} \mathbf{C} = \alpha \mathbf{S} \mathbf{P} \mathbf{P}^{-1} \mathbf{C} = \alpha \left[\begin{array}{c|c} \mathbf{I}_r & \\ \hline \mathbf{0}_{(m-r) \times 1} & \mathbf{S}' \end{array} \right] \left[\begin{array}{c|c} 1 & \\ \hline \mathbf{0}_{(r-1) \times 1} & \mathbf{C}' \end{array} \right]. \quad (8)$$

The reduced support matrix is $\mathbf{S} := \mathbf{S} \mathbf{P}$ and the reduced coefficient matrix is $\mathbf{C} := \mathbf{P}^{-1} \mathbf{C}$.

- $\mathbf{C}_{*,\{1..r\}} = \mathbf{I}_r$. By Gaussian elimination on rows of $\mathbf{C}$, there exists a matrix $\mathbf{Q} = \left[\begin{array}{c|c} 1 & \mathbf{q} \\ \hline \mathbf{0}_{(r-1) \times 1} & \mathbf{Q}' \end{array} \right] \in \text{GL}_r(\mathbb{F}_q)$ where $\mathbf{q} \in \mathbb{F}_q^{1 \times (r-1)}$ and $\mathbf{Q}' \in \text{GL}_{r-1}(\mathbb{F}_q)$ such that $\mathbf{S} \mathbf{Q}^{-1} = \left[\begin{array}{c|c} 1 & \\ \hline \mathbf{0}_{(m-r) \times 1} & \mathbf{S}' \end{array} \right]$ and $\mathbf{Q} \mathbf{C} = [\mathbf{I}_r \ \mathbf{C}']$ where $\mathbf{S}' \in \mathbb{F}_q^{m \times (r-1)}$ and $\mathbf{C}' \in \mathbb{F}_q^{r \times (n-r)}$. Then

$$\mathbf{e} = \alpha \mathbf{S} \mathbf{C} = \alpha \mathbf{S} \mathbf{Q}^{-1} \mathbf{Q} \mathbf{C} = \alpha \left[\begin{array}{c|c} 1 & \\ \hline \mathbf{0}_{(m-r) \times 1} & \mathbf{S}' \end{array} \right] [\mathbf{I}_r \ \mathbf{C}']. \quad (9)$$

The reduced support matrix is $\mathbf{S} := \mathbf{S} \mathbf{Q}^{-1}$ and the reduced coefficient matrix is $\mathbf{C} := \mathbf{Q} \mathbf{C}$.

Theorem 7.4. When $\mathbf{S}$ and $\mathbf{C}$ are reduced as in Equation (8), the OJ attack with the basis enumeration (OJ-S) solves the RD problems with complexity $\mathcal{O}((mN)^\omega q^{(m-r)(r-1)})$ over $\mathbb{F}_q$ where $N = \left\lceil \frac{kr}{m-r} \right\rceil$ and $N \leq n - k - 1$.

Theorem 7.5. When $\mathbf{S}$ and $\mathbf{C}$ are reduced as in Equation (9), the OJ attack with the coordinate enumeration (OJ-C) solves the RD problem with complexity $\mathcal{O}((mN)^\omega q^{(r-1)(N+k+1-r)})$ over $\mathbb{F}_q$ where $N = \left\lceil \frac{m(r-1)+k+1-r}{m-1} \right\rceil$ and $N \leq n - k - 1$.

7.2 Algebraic Attacks

Algebraic attacks on the RD problem have been extensively studied in [9–11, 26]. Among them, the most powerful approach is the Maximal Minors (MM) attack. Another powerful algebraic attack is the Support Minors (SM) attack, initially introduced in [11] for solving generic MinRank instances. This modeling has been further adapted to instances arising from $\mathbb{F}_{q^m}$ -linear codes in [10]. For the parameter sets considered in this submission (see Tables 6 and 7), the SM attack has a slightly higher complexity than the MM attack. From Tables 6 and 7, the challenge of mounting two attacks stems not just from its high time complexity, but from the fact that the required storage—at least $2^{35} \times 2^{35}$ binary matrix—is huge.

The MM Attack. By Equations (2), we have the equation $\varepsilon \mathbf{C} \mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$. This equation implies that $\mathbf{C} \mathbf{H}_y^\top \in \mathbb{F}_{q^m}^{r \times (n-k-1)}$ is not of row full rank because a non-zero vector ε belongs to its left kernel. Then all maximal minors $|\mathbf{C} \mathbf{H}_y^\top|_{*,J}$ of $\mathbf{C} \mathbf{H}_y^\top$ are equal to 0 for $J \subset \{1..n-k-1\}$ and $\#J = r$. By the Cauchy-Binet formula, each $|\mathbf{C} \mathbf{H}_y^\top|_{*,J}$ can be viewed as a non-zero linear combination about all non-zero maximal minors $c_T = |\mathbf{C}|_{*,T}$ for $T \subset \{1..n\}$ and $\#T = r$. The number of non-zero maximal minors c_T 's is $\binom{n}{r}$ and the number of non-zero linear combinations is $\binom{n-k-1}{r}$. This builds the MM modeling over $\mathbb{F}_{q^m}$:

$$\left\{ |\mathbf{C} \mathbf{H}_y^\top|_{*,J} = \sum_{T \subset \{1..n\}, \#T=r} c_T |\mathbf{H}_y^\top|_{T,J} = 0 : J \subset \{1..n-k-1\}, \#J = r \right\}. \quad (\text{MM-}\mathbb{F}_{q^m})$$

One wants to solve c_T 's from this linear system to determine $\mathbf{C}$. Once all non-zero c_T 's are solved from the MM modeling, one can determine the entries of $\mathbf{C}$. This point is associated with the Plücker coordinates in Grassmannian manifold (see Section 3 in [10] for a tailored description). Finally, one solves ε from the equation $\varepsilon \mathbf{C} \mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$ and determines $\mathbf{e} = \varepsilon \mathbf{C}$.

- **Linearization Technique:** One views c_T 's as unknowns, and builds a linear system with $\binom{n}{r}$ unknowns and $\binom{n-k-1}{r}$ equations over $\mathbb{F}_{q^m}$. One wants to solve c_T 's from this linear system to determine $\mathbf{C}$. However, the system has many solutions due to $\binom{n-k-1}{r} < \binom{n}{r}$, whereas one wants more equations than unknowns for a unique solution.
- **Unfolding Technique:** As $c_T \in \mathbb{F}_q$, to obtain more equations, one unfolds $\text{MM-}\mathbb{F}_{q^m}$ over $\mathbb{F}_q$, namely unfolds the coefficients, $|\mathbf{H}_{\mathbf{y}}^\top|_{T,J}$, over $\mathbb{F}_q$. In this way, one obtains $m\binom{n-k-1}{r}$ equations. In $\mathbb{F}_q$, the MM modeling ($\text{MM-}\mathbb{F}_q$) includes:

Unknowns: $\binom{n}{r}$ non-zero maximal minors $c_T \in \mathbb{F}_q$,

Equations: $m\binom{n-k-1}{r}$ independent equations over $\mathbb{F}_q$ in all non-zero c_T 's.

If $m\binom{n-k-1}{r} \geq \binom{n}{r} - 1$, the linear system experimentally has the rank $\binom{n}{r} - 1$, which implies a unique solution. If this case holds, by linearization, recovering all maximal minors consists in finding a vector in the right kernel of a matrix of size $m\binom{n-k-1}{r} \times \binom{n}{r}$ over $\mathbb{F}_q$. The final cost of solving $\text{RD}_{q,m,n,k,r}$ in $\mathbb{F}_q$ is given by

$$\mathcal{O} \left(m \binom{n-k-1}{r} \binom{n}{r}^{\omega-1} \right).$$

- **Puncturing Technique:** When $\text{MM-}\mathbb{F}_q$ is “super”-overdetermined: $m\binom{n-k-1}{r} \gg \binom{n}{r}$, one uses the puncturing technique to decrease equations. One punctures $\mathcal{C}$ on the last p coordinates while keeping dimension k . In this way, one obtains an instance of $\text{RD}_{q,m,n-p,k,r}$. Then it applies the MM modeling to the obtained instance such that the modeling remains overdetermined and the cost of solving the modeling is minimal.
- **Hybrid Technique:** When $\text{MM-}\mathbb{F}_q$ is underdetermined: $m\binom{n-k-1}{r} < \binom{n}{r}$, one uses the hybrid technique. Informally, one guesses that a positions of $\mathbf{e}$ are zeros, with successful probability q^{-ar} , such that these positions are included in an information set. In this way, one obtains an instance of $\text{RD}_{q,m,n-a,k-a,r}$. Then it applies the MM modeling to the obtained instance such that the modeling is overdetermined and the cost of solving the modeling is minimal.
- The complexity of solving the $\text{RD}_{q,m,n,k,r}$ problem by the MM modeling is estimated as

$$\begin{cases} \mathcal{O} \left(q^{ar} m \binom{n-k-1}{r} \left(\binom{n-a}{r} \right)^{\omega-1} \right), & m\binom{n-k-1}{r} < \binom{n}{r}, \\ \mathcal{O} \left(m \binom{n-p-k-1}{r} \left(\binom{n-p}{r} \right)^{\omega-1} \right), & m\binom{n-k-1}{r} \geq \binom{n}{r}, \end{cases}$$

where p is a positive integer such that $m\binom{n-p-k-1}{r} \geq \binom{n-p}{r} - 1$ holds and the complexity is minimal, and a is a positive integer such that $m\binom{n-k-1}{r} \geq \binom{n-a}{r} - 1$ holds and the complexity is minimal.

The overall complexity can also be summarized as: Combining the puncturing and hybrid techniques, solving $\text{RD}_{q,m,n,k,r}$ is transformed into solving $\text{RD}_{q,m,n-p-a,k-a,r}$, the overall complexity is

$$\mathcal{O} \left(q^{ar} m \binom{n-k-p-1}{r} \left(\binom{n-p-a}{r} \right)^{\omega-1} \right),$$

where p and a such that $m\binom{n-k-p-1}{r} \geq \binom{n-p-a}{r} - 1$ holds and the complexity is minimal.

- **Subsupport-Guessing Technique:** Recently, the Subsupport-Guessing (SG) technique was proposed in [20] to reduce the RD problem. The goal is to transform an underdetermined MM modeling into an overdetermined one. In the RD problem, let the error $\mathbf{e} = \boldsymbol{\varepsilon}\mathbf{C}$ where $\boldsymbol{\varepsilon} \in \mathbb{F}_{q^m}^r$ is $\mathbf{e}$'s basis and $\mathbf{C} \in \mathbb{F}_q^{r \times n}$ is $\mathbf{e}$'s coefficient matrix. In the SG technique, one guesses a random matrix $\mathbf{Q} \in \mathbb{F}_q^{n \times (n-h)}$ such that $\mathbf{Q}$'s h -dimensional left kernel is included in the r -dimensional row space of $\mathbf{C}$, which has a successful probability $q^{-h(n-r)}$. For an appropriate h ($0 \leq h \leq r-1$), right-multiplying the instance $(\mathbf{G}, \mathbf{y} = \mathbf{x}\mathbf{G} + \mathbf{e}, r)$ of $\text{RD}_{q,m,n,k,r}$ by $\mathbf{Q}$ results in the instance $(\mathbf{G}\mathbf{Q}, \mathbf{y}\mathbf{Q} = \mathbf{x}\mathbf{G}\mathbf{Q} + \mathbf{e}\mathbf{Q}, r-h)$ of $\text{RD}_{q,m,n-h,k,r-h}$. This is because $\mathbf{G}\mathbf{Q}$'s rank is still k with an overwhelming probability as long as $\text{rank}(\mathbf{Q}) > k$, $\mathbf{C}\mathbf{Q}$'s rank is $r-h$, and $\|\mathbf{e}\mathbf{Q}\|_R = \|\boldsymbol{\varepsilon}\mathbf{C}\mathbf{Q}\|_R = r-h$. With the SG technique, solving $\text{RD}_{q,m,n,k,r}$ is equivalent to solving $\text{RD}_{q,m,n-h,k,r-h}$ because $\mathbf{G}\mathbf{Q}$ keeps rank k and their instances share the same $\mathbf{x}$. The SG technique benefits from a small code length n , typically parameters region of signature RYDE [3]; otherwise, the guessing cost is very expensive.

Combining the puncturing, hybrid, and SG techniques, $\text{RD}_{q,m,n,k,r}$ is transformed into $\text{RD}_{q,m,n-p-a-h,k-a,r-h}$, the overall complexity can be summarized as a nonlinear programming:

Theorem 7.6. *Let p , a , and h be three integers. With the puncturing, hybrid, and SG techniques, the MM modeling solves $\text{RD}_{q,m,n,k,r}$ with a complexity over $\mathbb{F}_q$*

$$\mathcal{O} \left(q^{a(r-h)+h(n-h)} m \binom{n-k-p-h-1}{r-h} \binom{n-p-a-h}{r-h}^{\omega-1} \right),$$

where p , a , and h such that $m \binom{n-k-p-h-1}{r-h} \geq \binom{n-p-a-h}{r-h} - 1$ holds and the complexity is minimal.

The SM Attack. Recall that the received vector $\mathbf{y} = \mathbf{x}\mathbf{G} + \mathbf{e}$, where $\|\mathbf{e}\|_R = r$. Then, $\mathbf{e} = \mathbf{y} - \mathbf{x}\mathbf{G} = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_r)\mathbf{C}$ which implies that $\mathbf{y} - \mathbf{x}\mathbf{G} \in \mathbb{F}_{q^m}^n$ lies in the row space of $\mathbf{C}$. Consequently, all maximal minors of the matrix $\begin{bmatrix} \mathbf{y} - \mathbf{x}\mathbf{G} \\ \mathbf{C} \end{bmatrix}$ equal 0. By performing a Laplace expansion of the maximal minors along the first row, we obtain $\binom{n}{r+1}$ equations that are bilinear in the entries x_i of $\mathbf{x}$ (referred to as *linear variables*) and in the maximal minors c_T of $\mathbf{C}$. The SM modeling is defined as follows:

$$\left\{ Q_I := \left| \begin{bmatrix} \mathbf{y} - \mathbf{x}\mathbf{G} \\ \mathbf{C} \end{bmatrix} \right|_{*,I} : I \subset \{1..n\}, \#|I| = r+1 \right\} \quad (\text{SM-}\mathbb{F}_{q^m})$$

Unknowns: $\binom{n}{r}$ variables c_T defined over $\mathbb{F}_q$, k variables $x_1, x_2, \dots, x_k$ defined over $\mathbb{F}_{q^m}$.

Equations: $\binom{n}{r+1}$ equations $Q_I = 0$ for all $I \subset \{1..n\}$ with $\#I = r+1$. These equations are affine bilinear over $\mathbb{F}_{q^m}$, involving the variables x_i and the variables c_T .

As in the MM modeling, one solves c_T 's from this system to determine $\mathbf{C}$. Once all non-zero c_T 's are solved, one can determine the entries of $\mathbf{C}$. Finally, one solves $\boldsymbol{\varepsilon}$ from the equation $\boldsymbol{\varepsilon}\mathbf{C}\mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$ and determine $\mathbf{e} = \boldsymbol{\varepsilon}\mathbf{C}$. Or one solves x_i 's from this system to determine $\mathbf{x}$.

- **Linearization.** In $\text{SM-}\mathbb{F}_{q^m}$, an affine bilinear system, all bilinear monomials have the form of $x_i c_T$, whose number is $k \binom{n}{r}$. One applies linearization to transform the bilinear system to a linear system by replacing the monomial $x_i c_T$ by a single variable, then solves $x_i c_T$ from the resulting linear system, and further derives all c_T 's or x_i 's from all $x_i c_T$. If

there are more linearly independent equations than bilinear monomials, the linear system may be solved by linearization. Since cryptographic parameters always satisfy $\frac{n-r}{r+1} < k$, we have $\binom{n}{r+1} < k\binom{n}{r}$. Further, the system has many solutions due to $\binom{n}{r+1} < k\binom{n}{r}$, whereas one wants more equations than unknowns for a unique solution.

- **Decrease the number of c_T and equations.** The work [10] showed that the SM modeling contains the equations arising from the MM modeling (MM- $\mathbb{F}_q$). By reducing the system modulo MM- $\mathbb{F}_q$ and eliminating some variables c_T 's, we obtain a more compact SM modeling (SM- $\mathbb{F}_{q^m}^+$).

$$\text{SM-}\mathbb{F}_{q^m}^+ := \text{SM-}\mathbb{F}_{q^m} \bmod (\text{MM-}\mathbb{F}_q)$$

Unknowns: $\binom{n}{r} - m\binom{n-k-1}{r}$ variables c_T over $\mathbb{F}_q$, k variables $x_1, x_2, \dots, x_k$ over $\mathbb{F}_{q^m}$.

Equations: $\sum_{i_1=1}^k \sum_{i_2=i_1+1}^{k+1} \binom{n-i_2}{r-1} = \sum_{i=1}^k \binom{n-i}{r} - k\binom{n-k-1}{r} = \binom{n}{r+1} - \binom{n-k-1}{r+1} - (k+1)\binom{n-k-1}{r}$ equations of the form $\tilde{Q}_I = 0$, where $I = \{i_1, i_2, \dots, i_{r+1}\} \subset \{1..n\}$ satisfies $\#I = r+1$, $\#(I \cap \{1..k+1\}) \geq 2$, and $\tilde{Q}_I = Q_I \bmod (\text{MM-}\mathbb{F}_q)$ denotes the equation obtained from Q_I after eliminating the variables c_T using the MM- $\mathbb{F}_q$ relations.

Let $A := k\binom{n}{r} - km\binom{n-k-1}{r}$ and $B := \binom{n}{r+1} - \binom{n-k-1}{r+1} - (k+1)\binom{n-k-1}{r}$. Then

$$A > B \iff (n-r+k(r+1)-km(r+1))\binom{n-k-1}{r} > (n-r-k(r+1))\binom{n}{r}$$

For cryptographic parameters, we always have $A > B$, thus the system still has many solutions.

- **Multiply the equations by x_i^{b-1} .** To increase the number of equations, multiply the equations by all monomials x_i^{b-1} to derive a new system with monomials in the form of $x_i^b c_T$. The parameter b is chosen large enough so that equations are more than monomials and the linearization process can recover all monomials uniquely. The work [10] derived the number of (independent) equations and unknowns:

Unknowns: $U(q, m, n, k, r, b) = \binom{k+b-1}{b} \left(\binom{n}{r} - m\binom{n-k-1}{r} \right)$.

Equations: $E'(m, n, k, r, b) = \sum_{i_1=1}^k \sum_{i_2=i_1+1}^{k+1} \binom{n-i_2}{r-1} \binom{k-i_1+b-1}{b-1}$

Independent Syzygies: $S(m, n, k, r, b) = (m-1) \sum_{i=1}^b (-1)^{i+1} \binom{k+b-i-1}{b-i} \binom{n-k-1}{r+i}$

Independent Equations: $E(m, n, k, r, b) = E'(m, n, k, r, b) - S(m, n, k, r, b)$.

If there exists a b such that $E(m, n, k, r, b) \geq U(m, n, k, r, b) - 1$, the linear system experimentally has rank $U(m, n, k, r, b) - 1$, which implies a unique solution. By linearization, recovering all monomials consists in finding a vector in the right kernel of a matrix in $\mathbb{F}_{q^m}^{U \times E}$. In this case, the final cost of solving $\text{RD}_{q,m,n,k,r}$ in $\mathbb{F}_q$ is given by $\mathcal{O}(m^2 EU^{\omega-1})$.

Combining the puncturing, hybrid, and SG techniques, $\text{RD}_{q,m,n,k,r}$ is transformed into $\text{RD}_{q,m,n-p-a-h,k-a,r-h}$, the overall complexity can also be summarized as an integer nonlinear programming:

Theorem 7.7. *Let b, p, a , and h be four integers. Let $E := E(m, n-p-a-h, k-a, r-h, b)$ and $U := U(m, n-p-a-h, k-a, r-h, b)$. With the puncturing, hybrid, and SG techniques, the SM modeling solves $\text{RD}_{q,m,n,k,r}$ with a complexity over $\mathbb{F}_q$*

$$\mathcal{O}\left(q^{a(r-h)+h(n-h)} EU^{\omega-1}\right),$$

where b, p, a , and h such that $E \geq U - 1$ holds and the complexity is minimal.

7.3 Resistance to Quantum Attacks

Combinatorial Attacks: The overall complexity can be decomposed into a polynomial component and an exponential component. The polynomial part is dominated by solving a linear system, while the exponential part arises from guessing special subspaces of the ambient vector space or the support/coefficient matrix. In NIST PQC submissions RQC [33], it is commonly assumed that the guessing phase can be accelerated by a quadratic speedup using Grover’s algorithm. This corresponds to keeping the polynomial component unchanged while reducing the exponential component by a square-root factor. In this work, we adopt a more optimistic assumption that the polynomial component may also benefit from a quadratic speedup. Under this assumption, we estimate the quantum time complexity as the square root of the overall classical complexity.

Algebraic Attacks: The best algebraic attack is the MM attack, whose complexity primarily depends on solving a huge linear system and guessing certain entries of the coefficient matrix. The guessing phase can, in principle, be accelerated by a quadratic speedup using Grover’s algorithm. The resulting linear system is typically very large and exhibits approximately moderate density over finite fields. When the MM modeling is overdetermined, the overall complexity is dominated by the cost of solving this linear system, which is far from negligible. Moreover, unlike the regular structure of guessing operations, the irregular nature of linear system solving may prevent achieving a straightforward quadratic speedup. As noted in the NIST PQC submissions RYDE [3] and Mirath [2], such a square-root acceleration cannot be directly applied. To date, no specialized quantum algorithm is known for efficiently solving linear systems over finite fields.

- One candidate quantum approach is the Harrow–Hassidim–Lloyd (HHL) algorithm. However, HHL is designed for sparse systems over real or complex fields. Its complexity is given by $\mathcal{O}\left(\log N \cdot \frac{\tau^2 \kappa^2}{\varepsilon}\right)$, where N is the size of matrix, τ is sparsity, κ is condition number, and ε the precision of solutions ($0 \leq \varepsilon \leq 1$, where smaller values of ε correspond to higher precision). When attempting to apply this framework to finite fields, fundamental issues arise: the condition number κ is not well-defined, and exact solutions require $\varepsilon = 0$, which falls outside the algorithm’s assumptions.

Unlike Grover’s algorithm, which has been well established for accelerating guessing procedures, there is currently no concrete evidence supporting similar speedups for solving huge linear systems of approximately moderate density over finite fields. Nevertheless, given the current uncertainty surrounding quantum algorithms and hardware, we adopt an optimistic assumption that solving such linear systems may also benefit from a quadratic speedup. Under this assumption, we estimate the quantum time complexity as the square root of the overall classical complexity, in line with standard heuristics used for combinatorial attacks.

7.4 Estimated Security Strength

In this section, by combinatorial and algebraic attacks recalled in Sections 7.1 and 7.2, we estimate security strength for the parameter sets proposed in Table 2.

We first want to say that **our estimations are conservative.**

1. **The linear algebra constant ω is conservative and is set as 2.** All attacks against the RD problem involve solving a linear system. Typically, the most powerful MM algebraic attack, the complexity is completely determined by solving a linear system when

the MM modeling is overdetermined. Solving a linear system amounts to finding a vector belonging to the right kernel of its matrix. Recall that for a $\mu \times \nu$ matrix of rank ρ over $\mathbb{F}_q$, the cost to find a vector in its right kernel is $\mathcal{O}(\rho^{\omega-2}\mu\nu)$ operations in $\mathbb{F}_q$ [17, 29, 41], where ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$. The cost is determined by the linear algebra constant ω . The value ω usually is considered to be 3. Precisely, ω depends on the specific matrices and algorithms.

- The Strassen algorithm [43]. It is a *practical* algorithm for any matrix. If the matrix contains more than a few hundred rows and columns, a practical ω is 2.807.
- The Coppersmith-Winograd algorithm [19] and its improvements [28, 42, 45]. This type of algorithms is based on the laser method introduced by Strassen [44]. In [28], ω is optimized to about 2.373, which is currently best known value. However, they are considered to be only *theoretical* and *impractical* due to at least an extremely large constant factor.
- The Wiedemann algorithm [47]. It is a *practical* algorithm for a *sparse* matrix. If the matrix only has τ non-zero terms per row, then the cost is given by $\mathcal{O}(\tau\mu\nu)$. If $\tau = \mathcal{O}(1)$, the cost is the same as one using $\omega = 2$.

Overall, it is widely held that the practical ω is about 2.807 and the theoretical lower bound approaches 2. However, the development of a practical algorithm that achieves the lower bound 2 while maintaining a tractable constant factor is a very *long-term* challenge in computational theory. At least, for a huge size or moderately dense matrix, a practical ω should still be 2.807. For the most powerful MM attack, the induced linear systems usually have a huge size and (approximately) moderate density. Nevertheless, for security parameters in Table 2, we conservatively set $\omega = 2$. This setting also takes into account any algorithm that could take advantage of the structure of the matrices.

2. **The classical time complexity redundancy is conservative.** We reserve the complexity redundancy of at least 18 bits. We refer to Table 6, Table 7, and Table 8.
3. **The quantum time complexity is conservative.** As described in Section 7.3, we make more optimistic assumptions than the NIST PQC submissions [2, 3, 33] for the estimation of the quantum time complexity. We estimate the quantum time complexity as the square root of the classical time complexity (including solving a linear system).
4. **The space complexity is conservative.** While the MM attack leads to the lowest time complexity, it is difficult to devise such an attack because the induced linear system is very huge. It requires a massive memory, about 2^{71} memory for the time complexity of 2^{170} . We only consider the space complexity as the memory required to store a linear system. We refer to Table 6.

In Tables 6 - 8, we compute the complexity of the best attacks on the RSD problem. In Table 6, we compute the (time and space) complexity of the MM attack, which is the best classical attack, and one can verify the complexity by the optimal a , p , and h . In Table 7, we compute the (time and space) complexity of the SM algebraic attack, which one can verify the complexity by the optimal b , a , p , and h . In Table 8, we compute the classical time complexity of the combinatorial attacks such as the AGHT and OJ attacks. “Attack- w_1 ” represents attacking the $\text{RSD}_{q,m,2n,n,w_1}$ problem. “Attack- w_2 ” represents attacking the $\text{RSD}_{q,m,2n+N_1,n,w_2}$ problem. The complexity of these attacks is recalled in Section 7.

The complexities estimated in the table are for solving the RSD problem. As reviewed in Section 2.3 for the RSL problem, its complexity under combinatorial attacks is approximately

Table 6: Classical complexity of the **MM** algebraic attack

Instance	Attack- w_1 ($\xi, a, p, h, \xi', \text{TC}, \text{SC}$)	Attack- w_2 ($\xi, a, p, h, \xi', \text{TC}, \text{SC}$)	Minimal TC
Level-128	(0.14, 14, 0, 0, 1.01, 2^{172} , 2^{60})	(0.27, 11, 0, 0, 1.08, 2^{170} , 2^{71})	2^{170}
Level-256	(0.06, 22, 0, 0, 1.08, 2^{296} , 2^{76})	(0.14, 17, 1, 0, 1.03, 2^{276} , 2^{89})	2^{276}
Level-512	(0.02, 36, 1, 0, 1.00, 2^{530} , 2^{98})	(0.03, 33, 0, 0, 1.00, 2^{580} , 2^{118})	2^{530}

“ ξ ” (resp. “ ξ' ”) represents the (resp. updated) rate of equations and unknowns. “TC” represents the time complexity of the MM attack. “SC” represents the space complexity of the MM attack, which is computed as the memory required to store a linear system. We conservatively estimate “SC” as the size of a linear system.

Table 7: Classical complexity of the **SM** algebraic attack

Instance	Attack- w_1 ($\xi, b, a, p, \xi', \text{TC}, \text{SC}$)	Attack- w_2 ($\xi, b, a, p, \xi', \text{TC}, \text{SC}$)	Minimal TC
Level-128	(0.21, 1, 14, 1, 1.19, 2^{187} , 2^{62})	(0.24, 1, 10, 0, 1.67, 2^{176} , 2^{73})	2^{176}
Level-256	(0.17, 1, 21, 0, 1.32, 2^{302} , 2^{78})	(0.19, 1, 16, 0, 3.07, 2^{280} , 2^{89})	2^{280}
Level-512	(0.14, 1, 35, 0, 2.20, 2^{535} , 2^{100})	(0.14, 2, 32, 0, 1.19, 2^{593} , 2^{130})	2^{535}

“ ξ ” (resp. “ ξ' ”) represents the (resp. updated) rate of equations and unknowns. “TC” represents the time complexity of the SM attack. “SC” represents the space complexity of the SM attack, which is computed as the memory required to store a linear system. We conservatively estimate “SC” as the size of a linear system. The optimal $h = 0$.

Table 8: Classical time complexity of the **AGHT** and **OJ** combinatorial attacks

Instance	Attack- w_1 (AGHT, OJ)	Attack- w_2 (AGHT, OJ)	Minimal TC
Level-128	(2^{271} , 2^{270})	(2^{268} , 2^{306})	2^{268}
Level-256	(2^{474} , 2^{434})	(2^{514} , 2^{480})	2^{434}
Level-512	(2^{831} , 2^{770})	(2^{965} , 2^{906})	2^{770}

We only compute the time complexity (TC). The space complexity is about the memory required to store a linear system, namely the polynomial part of the complexity formula, which is negligible to the MM attack.

that of solving RSD reduced by N bits, while the complexity of algebraic attacks remains unchanged. In this case, algebraic attacks remain more powerful than combinatorial attacks and thus determine the security strength of the algorithm. The complexity of combinatorial attacks on the RSL problem is estimated in Table 9.

Table 9: Classical time complexity of the **AGHT** and **OJ** combinatorial attacks on **RSL**

Instance	Attack- w_1-N_1 (AGHT, OJ)	Attack- w_2-N_2 (AGHT, OJ)	Minimal TC
Level-128	(2^{260} , 2^{259})	(2^{256} , 2^{290})	2^{256}
Level-256	(2^{459} , 2^{415})	(2^{498} , 2^{464})	2^{415}
Level-512	(2^{812} , 2^{751})	(2^{944} , 2^{885})	2^{751}

“Attack- w_1-N_1 ” represents attacking the $\text{RSL}_{q,m,2n,n,w_1,N_1}$ problem. “Attack- w_2-N_2 ” represents attacking the $\text{RSL}_{q,m,2n+N_1,n,w_2,N_2}$ problem. The values N_1 and N_2 are given in Table 2.

8 Side-Channel Attacks

Currently, no tailored side-channel attack exists against C-Multi-UR-AG. The side-channel attacks on RQC and the RD problem can be adapted to C-Multi-UR-AG.

The resistance of RQC to timing attacks, as studied in [15], can be adapted to C-Multi-UR-AG. These timing attacks exploit a correlation between the weight of the error to be decoded and the running time of the decoding algorithm. While of theoretical interest, such attacks are quite impractical in real-world scenarios as they require a huge number of queries to a timing oracle. Moreover, the reference and optimized implementations are designed to avoid leaking the error weight, thus preventing these timing attacks.

The works [21, 36] analyzed side-channel attacks on the RD problem, showing that partial leakage of information about the solution can weaken the problem's hardness. However, the critical premise lies in how such information about the solution can be obtained. In line with the timing attacks discussed above, our implementations are designed to avoid the leakage of information about the solution.

9 Advantages and Limitations

Advantages. The main advantages are:

- The security is only reduced to a well-understood problem in coding theory: the syndrome decoding problem.
- The security immunity against attacks recovering the hidden structure of a specific code.
- It features a small public key size.
- It features a precise DFR.
- It features a clean and clear structure.
- It features a small bandwidth among code-based unstructured KEMs, with the same linear algebra constant ω and complexity redundancy.
- It features a smaller bandwidth than the lattice-based unstructured KEMs, namely FrodoKEM [1] and Scloud⁺ [46].

Limitations. The main limitations are:

- The rank metric for cryptosystems may still not be old (1991). Although this might be viewed as a limitation, there also exist further activities on understanding the inherent computational complexity of the related problems. Nevertheless, the combinatorial attacks have been very well understood, and the recent works [5, 38, 40] allow a clear view on the complexity of algebraic attacks, giving conservative estimations of complexity. Moreover, we have selected very conservative parameters for potential attacks.
- The decoding and properties of the AG codes are recently developed [16, 38], while its concept can be traced back to 2009 [13]. Its decoding performance should be further improved for the efficiency of cryptosystems.

References

- [1] Alkim, E., Bos, J.W., Ducas, L., et al.: FrodoKEM: Learning with errors key encapsulation - preliminary standardization proposal (2024), <https://frodokem.org/> 29
- [2] Aragon, G.A., Aragon, N., Barbero, S., et al.: Mirath. Second round submission to the NIST post-quantum cryptography call: additional digital signature schemes (2025), <https://pqc-mirath.org/> 26, 27
- [3] Aragon, N., Bardet, M., Bidoux, L., et al.: RYDE. Second round submission to the NIST post-quantum cryptography call: additional digital signature schemes (2025), <https://pqc-ryde.org/> 24, 26, 27
- [4] Aragon, N., Bettaieb, S., Bidoux, L., Connan, Y., Coulaud, J., Gaborit, P., Kominiaz, A.: The rank-based cryptography library. In: International Workshop on Code-Based Cryptography (CBCrypto). vol. 13150, pp. 22–41. Springer (2021) 14
- [5] Aragon, N., Briaud, P., Dyseryn, V., Gaborit, P., Vinçotte, A.: The blockwise rank syndrome learning problem and its applications to cryptography. In: Post-Quantum Cryptography (PQCrypto). vol. 14771, pp. 75–106. Springer (2024) 18, 29
- [6] Aragon, N., Gaborit, P., Hauteville, A., Tillich, J.: A new algorithm for solving the rank syndrome decoding problem. In: International Symposium on Information Theory (ISIT). pp. 2421–2425. IEEE (2018) 18
- [7] Augot, D., Loidreau, P., Robert, G.: Generalized Gabidulin codes over fields of any characteristic. Designs, Codes and Cryptography 86(8), 1807–1848 (2018) 3, 10
- [8] Bardet, M., Briaud, P.: An algebraic approach to the rank support learning problem. In: Post-Quantum Cryptography (PQCrypto). vol. 12841, pp. 442–462. Springer (2021) 5, 13
- [9] Bardet, M., Briaud, P., Bros, M., Gaborit, P., Neiger, V., Ruatta, O., Tillich, J.: An algebraic attack on rank metric code-based cryptosystems. In: Advances in Cryptology - EUROCRYPT. vol. 12107, pp. 64–93. Springer (2020) 22
- [10] Bardet, M., Briaud, P., Bros, M., Gaborit, P., Tillich, J.: Revisiting algebraic attacks on MinRank and on the rank decoding problem. Designs, Codes and Cryptography 91(11), 3671–3707 (2023) 22, 25
- [11] Bardet, M., Bros, M., Cabarcas, D., Gaborit, P., Perlner, R.A., Smith-Tone, D., Tillich, J., Verbel, J.A.: Improvements of algebraic attacks for solving the rank decoding and MinRank problems. In: Advances in Cryptology - ASIACRYPT. vol. 12491, pp. 507–536. Springer (2020) 22
- [12] Berger, T.P., Ourivski, A.: Construction of new MDS codes from Gabidulin codes. In: Proceedings of ACCT. pp. 40–47 (2009) 7, 9
- [13] Berger, T.P., Ourivski, A.: Construction of new MDS codes from Gabidulin codes. In: Proceedings of ACCT. pp. 40–47 (2009) 9, 29
- [14] Berlekamp, E.R., McEliece, R.J., van Tilborg, H.C.A.: On the inherent intractability of certain coding problems (corresp.). IEEE Transactions on Information Theory 24(3), 384–386 (1978) 4

- [15] Bettaieb, S., Bidoux, L., Gaborit, P., Marcatel, E.: Preventing timing attacks against RQC using constant time decoding of gabidulin codes. In: International Conference on Post-Quantum Cryptography (PQCrypto). vol. 11505, pp. 371–386. Springer (2019) 29
- [16] Bidoux, L., Briaud, P., Bros, M., Gaborit, P.: RQC revisited and more cryptanalysis for rank-based cryptography. *IEEE Transactions on Information Theory* **70**(3), 2271–2286 (2024) 3, 5, 7, 9, 13, 29
- [17] Bunch, J.R., Hopcroft, J.E.: Triangular factorization and inversion by fast matrix multiplication. *Mathematics of Computation* **28**(125), 231–236 (1974) 27
- [18] Buss, J.F., Frandsen, G.S., Shallit, J.O.: The computational complexity of some problems of linear algebra. *Journal of Computer and System Sciences* **58**(3), 572–596 (1999) 4
- [19] Coppersmith, D., Winograd, S.: Matrix multiplication via arithmetic progressions. In: Symposium on Theory of Computing (STOC). pp. 1–6. ACM (1987) 27
- [20] Couvée, H.B.S., Wachter-Zeh, A., Weger, V.: Hybrid subsupport guessing: A new hybrid technique for the rank decoding problem. *Cryptology ePrint Archive*, Paper 2025/2168 (2025), <https://eprint.iacr.org/2025/2168> 24
- [21] D’Alconzo, G., Esser, A., Gangemi, A., Sanna, C.: Sneaking up the ranks: Partial key exposure attacks on rank-based schemes. *Design, Codes and Cryptography* **94**(1), 15 (2026) 18, 20, 29
- [22] Debris-Alazard, T., Tillich, J.: Two attacks on rank metric code-based schemes: RankSign and an IBE Scheme. In: *Advances in Cryptology - ASIACRYPT*. vol. 11272, pp. 62–92. Springer (2018) 5, 13
- [23] Faugère, J., Levy-dit-Vehel, F., Perret, L.: Cryptanalysis of MinRank. In: *Advances in Cryptology - CRYPTO*. vol. 5157, pp. 280–296. Springer (2008) 4
- [24] Gabidulin, E.M.: Theory of codes with maximum rank distance. *Problemy peredachi informatsii* **21**(1), 3–16 (1985) 7
- [25] Gaborit, P., Hauteville, A., Phan, D.H., Tillich, J.: Identity-based encryption from codes with rank metric. In: *Advances in Cryptology - CRYPTO*. vol. 10403, pp. 194–224. Springer (2017) 5, 13
- [26] Gaborit, P., Ruatta, O., Schrek, J.: On the complexity of the rank syndrome decoding problem. *IEEE Transactions on Information Theory* **62**(2), 1006–1019 (2016) 18, 22
- [27] Gaborit, P., Zémor, G.: On the hardness of the decoding and the minimum distance problems for rank codes. *IEEE Transactions on Information Theory* **62**(12), 7245–7252 (2016) 4
- [28] Gall, F.L.: Powers of tensors and fast matrix multiplication. In: *International Symposium on Symbolic and Algebraic Computation (ISSAC)*. pp. 296–303. ACM (2014) 27
- [29] von zur Gathen, J., Gerhard, J.: *Modern Computer Algebra*. Cambridge University Press (2013) 27
- [30] Glabush, L., Hövelmanns, K., Stebila, D.: On the multi-target security of post-quantum key encapsulation mechanisms. *IACR Communications in Cryptology* **3**(1), 20 (2026) 7, 12, 17

- [31] Loidreau, P.: A Welch-Berlekamp like algorithm for decoding Gabidulin codes. In: International Workshop on Coding and Cryptography (WCC). vol. 3969, pp. 36–45. Springer (2005) [3](#)
- [32] Loidreau, P.: Decoding rank errors beyond the error-correcting capability. In: International Workshop on Algebraic and Combinatorial Coding Theory (ACCT). pp. 168–190 (2006) [7](#)
- [33] Melchor, C.A., Aragon, N., Bettaieb, S., et al.: RQC. Second round submission to the NIST post-quantum cryptography call (2020), <http://pqc-rqc.org/> [26](#), [27](#)
- [34] Ore, O.: On a special class of polynomials. Transactions of the American Mathematical Society **35**(3), 559–584 (1933) [7](#), [8](#)
- [35] Ourivski, A.V., Johansson, T.: New technique for decoding codes in the rank metric and its cryptography applications. Problems of Information Transmission **38**(3), 237–246 (2002) [20](#)
- [36] Porwal, A., Banda, H., Brinkmann, J., Baumeister, A., Krämer, J., Wachter-Zeh, A.: Subspace guessing and rank-metric solvers with hints. IACR Cryptology ePrint Archive p. 132 (2026), <https://eprint.iacr.org/2026/132> [29](#)
- [37] Song, Y., Chen, R., Huang, X., Zhang, J., Lin, C.: Refined OJ attacks: Tight complexity for rank decoding problems and their cryptographic implications. Cryptology ePrint Archive, Paper 2026/1291 (2026), <https://eprint.iacr.org/2026/1291> [20](#)
- [38] Song, Y., Chen, R., Zhang, F., Huang, X., Weng, J., Wang, H.: (Interleaved) extended Gabidulin codes, more analysis on blockwise rank decoding problem, and their applications to cryptosystems. Cryptology ePrint Archive, Paper 2025/668 (2025), <https://eprint.iacr.org/2025/668> [3](#), [9](#), [10](#), [29](#)
- [39] Song, Y., Zhang, J., Huang, X., Wu, W.: Blockwise rank decoding problem and LRPC codes: Cryptosystems with smaller sizes. In: Advances in Cryptology - ASIACRYPT. vol. 14444, pp. 284–316. Springer (2023) [18](#)
- [40] Song, Y., Zhang, J., Huang, X., Wu, W.: Blockwise rank decoding problem and LRPC codes: Cryptosystems with smaller sizes. IEEE Transactions on Information Theory **71**(6), 4806–4838 (2025) [18](#), [29](#)
- [41] Storjohann, A.: Algorithms for matrix canonical forms. Ph.D. thesis, ETH Zurich (2000) [27](#)
- [42] Stothers, A.J.: On the complexity of matrix multiplication. Ph.D. thesis, The University of Edinburgh (2010) [27](#)
- [43] Strassen, V.: Gaussian elimination is not optimal. Numerische Mathematik **13**(4), 354–356 (1969) [27](#)
- [44] Strassen, V.: The asymptotic spectrum of tensors and the exponent of matrix multiplication. In: Annual Symposium on Foundations of Computer Science (FOCS). pp. 49–54. IEEE Computer Society (1986) [27](#)
- [45] Vassilevska Williams, V.: Multiplying matrices faster than Coppersmith-Winograd. In: Symposium on Theory of Computing Conference (STOC). pp. 887–898. ACM (2012) [27](#)

- [46] Wang, A., Zheng, Z., Zhao, C., Qiu, Z., Zeng, G., Yuan, Y., Mu, C., Wang, X.: Scloud⁺: An efficient LWE-based KEM without ring/module structure. In: International Conference on Security Standardisation Research (SSR). vol. 15559, pp. 147–174. Springer (2024) [29](#)
- [47] Wiedemann, D.H.: Solving sparse linear equations over finite fields. IEEE Transactions on Information Theory **32**(1), 54–62 (1986) [27](#)